sora2-video-generation-flow

Sora 2 连续性视频生成：完整流程详解

贯穿全文的例子

假设用户写了一段短剧创意，最终 AI 拆出 **6 个分镜**，用户选了 **10 秒模式**：

代码块

```
分镜1：男主推门走入房间，环顾四周          4秒
分镜2：女主惊讶回头，手中针线掉落          3秒
分镜3：两人四目相对，空气凝固             5秒
分镜4：男主开口说"我回来了"               4秒
分镜5：女主眼眶泛红，嘴唇微颤             3秒
分镜6：男主上前拥抱女主                   3秒
```

---

① SCRIPT_PLANNER — 剧本大纲

AI 调用：文本 AI × 1 次

用户在 PROMPT_INPUT 节点输入创意（比如"一个仙侠短剧，男主离家三年后归来...")，传给 SCRIPT_PLANNER 节点。

Gemini 根据创意生成完整的剧本大纲，产出以下关键字段：

字段	值（示例）	作用
`scriptOutline`	完整大纲文本（世界观、章节、角色）	作为后续所有节点的上下文
`scriptVisualStyle`	`"3D"` / `"REAL"` / `"ANIME"`	**向下传播到所有下游节点的风格前缀**
`scriptGenre`	`"仙侠"`	类型标签，后续构建提示词时使用
`scriptSetting`	`"古代中国，修仙世界"`	背景设定

这个节点最关键的产出是 `scriptVisualStyle`。它决定了整部短剧的画面风格：

- `"3D"` → 所有提示词前缀变为 "Xianxia 3D animation character, semi-realistic style..."

- `"REAL"` → 变为 "Professional portrait photography, photorealistic human..."
- `"ANIME"` → 变为 "Anime character, anime style, 2D anime art..."

这个风格信息不是在这个节点使用的，而是通过 `getUpstreamStyleContext()` 函数被下游节点（③⑤⑦）递归查找上来使用的。

代码实现架构

1. 入口函数

函数名：`generateScriptPlanner`

代码块

```
async function generateScriptPlanner(
    prompt: string,        // 用户输入的创意文本
    config: {              // 用户在节点 UI 中填写的配置
        theme?: string,          // 主题，如 "爱情与成长"
        genre?: string,          // 类型，如 "仙侠"
        setting?: string,        // 背景，如 "古代中国，修仙世界"
        episodes?: number,       // 总集数，如 10
        duration?: number,       // 单集时长（分钟），如 1
        visualStyle?: string     // 视觉风格，"3D" / "REAL" / "ANIME"
    },
    refinedInfo?: Record<string, string[]>,  // 可选，来自剧目精炼节点的参考信息
    model?: string,                          // 可选，指定使用的 AI 模型
    context?: { nodeId?: string; nodeType?: string }  // 调用上下文（用于日志）
): Promise<string>    // 返回 AI 生成的 Markdown 格式剧本大纲文本
```

2. 第一步：动态参数计算

根据用户设定的集数，自动计算章节数、角色数、物品数的上下限：

代码块

```
const episodes = config.episodes || 10;                    // 总集数，默认10
const episodesPerChapter = 4;                               // 每章固定4集
const chapterCount = Math.ceil(episodes / episodesPerChapter);  // 章节数
const minCharacters = Math.round(10 + (episodes * 0.15));  // 最少角色数
const maxCharacters = Math.round(minCharacters * 1.3);     // 最多角色数
const minItems = Math.round(8 + (episodes * 0.1));         // 最少物品数
const maxItems = Math.round(minItems * 1.25);              // 最多物品数
```

计算结果对照表：

用户设定集数	章节数	角色数范围	物品数范围
10 集	3 章	12-16 人	9-11 个
30 集	8 章	15-19 人	11-14 个
50 集	13 章	18-23 人	13-16 个
100 集	25 章	25-33 人	18-23 个

3. 第二步：构建系统提示词（systemInstruction）

系统提示词是一个约 200 行的固定模板，包含 8 个占位符，在这一步被替换为实际数值：

代码块

```
let systemInstruction = SCRIPT_PLANNER_INSTRUCTION
    .replace(/{TotalEpisodes}/g, String(episodes))          // 总集数 → "10"
    .replace(/{ChapterCount}/g, String(chapterCount))       // 章节数 → "3"
    .replace(/{EpisodesPerChapter}/g, String(4))            // 每章集数 → "4"
    .replace(/{MinCharacters}/g, String(minCharacters))     // 最少角色 → "12"
    .replace(/{MaxCharacters}/g, String(maxCharacters))     // 最多角色 → "16"
    .replace(/{MinItems}/g, String(minItems))               // 最少物品 → "9"
    .replace(/{MaxItems}/g, String(maxItems))               // 最多物品 → "11"
    .replace(/{Duration}/g, String(config.duration || 1));  // 单集时长 → "1"
```

模板的完整结构如下：

代码块

```
你是一位专精于短剧和微电影的专业编剧 (Professional Screenwriter)。
你的任务是根据用户的核心创意和约束条件，创建一个引人入胜的**中文剧本大纲**。

核心原则：剧本大纲只在章节层面规划，不细化到每集

如果提供了【参考信息 - 来自剧目精炼】：
- 这些信息仅作为创作参考，用于启发灵感和确定方向
- 不要照搬，而是将其作为创作风格、主题方向的参考
- 可以借鉴其中的情感基调、受众定位、角色特征等元素
- 最终作品应该是原创的，融合用户的核心创意和你的专业创作

---

## 📊 剧集规模要求

本剧为 **{TotalEpisodes} 集**，需要规划 **{ChapterCount} 个章节**，
每个章节包含 **{EpisodesPerChapter} 集**。
```

角色数量要求：{MinCharacters}-{MaxCharacters} 个角色

角色分级与描述重点：

A. 核心角色（3-5人）- 需要详细小传
   - 主角团队（2-3人）：故事的绝对核心
   - 核心反派（1-2人）：与主角对抗的主要力量
   描述要求（每个角色80-120字）：
   > 林萧 - 男主，24岁，表面冷漠的实习医生...
     性格：外冷内热...外貌：黑发黑瞳...

B. 重要配角（8-12人）- 简单描述
   描述要求（每个角色20-40字）：
   > 苏晴 - 女主的闺蜜，性格活泼开朗...

C. 其他角色（剩余数量）- 一笔带过
   描述要求（每个角色5-10字）：
   > 护士小李 - 医院护士，配角。

物品数量要求：{MinItems}-{MaxItems} 个关键物品

物品分级与描述：

A. 核心物品（3-5个）- 推动主线
   描述要求（每个物品30-50字）

B. 辅助物品（5-8个）- 特定章节使用
   描述要求（每个物品15-25字）

C. 世界物品（剩余数量）- 丰富设定
   描述要求（每个物品10-15字）

🎯 章节结构与节奏要求

核心原则：每章包含2-5集，描述这几集的整体故事

节奏规律（必须严格遵循）：

1. 小高潮：每3-5集设置一次小高潮
   第3-5集：第一次小高潮
   第8-10集：第二次小高潮
   第13-15集：第三次小高潮
   ...

2. 大转折：每10-15集设置一次大转折
   第10-15集：剧情大反转/阵营重组/重大秘密揭露
   第20-25集：中期转折/角色关系巨变
   第30-35集：后期转折/核心冲突升级
   ...

📝 输出格式要求（必须严格遵守 Markdown 格式）

剧名（Title）
一句话梗概（Logline）：[一句话总结故事核心]
类型（Genre）：[类型] | **主题（Theme）**：[主题] |
背景（Setting）：[故事背景] | **视觉风格**：[Visual Style]

主要人物小传

核心角色（详细小传，80-120字/人）
* **[姓名]**：[角色定位] - [年龄] [外貌特征]。
  性格：[性格特点]。背景：[重要经历]。能力/特征：[特殊能力或标志性特征]。

重要配角（简单描述，20-40字/人）
* **[姓名]**：[角色定位和作用，简短描述]

其他角色（一笔带过，5-10字/人）
* **[姓名]**：[身份或作用]

关键物品设定

核心物品（30-50字/个）
* **[物品名称]**：[物品描述、功能、象征意义]

辅助物品（15-25字/个）
* **[物品名称]**：[物品描述和出现时机]

世界物品（10-15字/个）
* **[物品名称]**：[简要描述]

剧集结构规划（共 {TotalEpisodes} 集，{ChapterCount} 章）

章节格式标准（每章100-150字）

第X章：章节名称（第A-B集）

涉及角色：[本章主要角色，3-5人]
关键物品：[本章重要物品，2-3个]
章节剧情（100-150字）：
[这几集的整体故事描述，包含起承转合]
- [第A集]：[发生了什么]
- [第A+1集]：[情节推进]
- [第B集]：[本章高潮/转折]
关键节点：[标注：小高潮 或 大转折]

章节示例

第一章：觉醒篇（第1-4集）

涉及角色：林萧、苏晴、王院长
关键物品：死神之眼徽章、神秘纸条
章节剧情（120字）：
实习医生林萧发现自己能看到他人的死亡倒计时，从恐惧到逐渐接受。
院长王建国察觉异常，暗中观察。林萧与护士苏晴建立信任，发现医院隐藏的密室。
神秘黑衣人出现，留下纸条"我知道你能看到什么"。
林萧意识到自己卷入巨大阴谋，与苏晴联手逃脱追捕。
第4集高潮：林萧首次主动使用能力，看到王建国只剩3天寿命。
关键节点：小高潮（第4集）

⚠️ 重要规则

1. 只在章节层面规划，不要细化到每集的具体场景
2. 每章100-150字，简洁描述这几集的核心内容
3. 每章必须包含明确的起承转合
4. 严格遵守节奏规律：3-5集小高潮，10-15集大转折
5. 章节之间要有因果链条，避免情节断裂
6. 角色描述按分级处理：核心角色详细，配角简单，其他一笔带过
7. 物品描述按分级处理：核心详细，辅助简略，世界物品最简
8. 物品名称必须统一，不要使用同义词
9. 单集时长参考：{Duration} 分钟
10. 在创作中请始终贯彻[Visual Style]的视觉美学
11. 如有参考信息，灵活运用但不要生搬硬套，保持原创性

📌 质量检查清单

```

在输出前，请确认：
- [ ] 章节数量 = {ChapterCount} 个
- [ ] 每章包含 {EpisodesPerChapter} 集
- [ ] 每章100-150字
- [ ] 核心角色有详细小传（80-120字/人）
- [ ] 重要配角有简单描述（20-40字/人）
- [ ] 其他角色有一笔带过（5-10字/人）
- [ ] 核心物品有详细描述（30-50字/个）
- [ ] 辅助物品有简略描述（15-25字/个）
- [ ] 世界物品有简单描述（10-15字/个）
- [ ] 每章有明确的起承转合
- [ ] 遵循3-5集小高潮、10-15集大转折规律
- [ ] 在关键节点处标注"小高潮"或"大转折"
- [ ] 章节之间有连贯性
- [ ] 全中文输出
```

4. 第三步：构建精炼信息参考文本（可选）

如果上游有剧目精炼节点（DRAMA_REFINED）传来的数据，需要把它格式化后拼入用户提示词。

精炼信息的数据结构是 `Record<string, string[]>`，即一个键值对，键是字段名，值是字符串数组。

支持的 8 个字段及其显示标签：

内部字段名	显示标签
`dramaIntroduction`	剧集介绍参考
`worldview`	世界观参考
`logicalConsistency`	逻辑自洽性参考
`extensibility`	延展性参考
`characterTags`	角色特征参考
`protagonistArc`	主角弧光参考
`audienceResonance`	受众共鸣参考
`artStyle`	画风参考

拼接逻辑：

```
代码块
```

```
let refinedReference = '';

if (refinedInfo 存在且不为空) {
    refinedReference = '\n\n【参考信息 - 来自剧目精炼】\n';
    refinedReference += '以下信息仅作为创作参考，不要完全照搬，而是融入你的创意中:\n\n';

    for (每个字段键值对) {
        refinedReference += `${显示标签}:\n`;
        for (每个值) {
            refinedReference += `  - ${值}\n`;
        }
        refinedReference += '\n';
    }
}
```

输出示例:

代码块

```
【参考信息 - 来自剧目精炼】
以下信息仅作为创作参考，不要完全照搬，而是融入你的创意中:

角色特征参考:
  - 热血
  - 逆袭

画风参考:
  - 国风水墨
```

5. 第四步：构建用户提示词（userPrompt）

将用户在节点 UI 中填写的所有配置项拼接成一个完整的用户提示词:

代码块

```
const fullPrompt = `
核心创意: ${prompt}
主题: ${config.theme || 'N/A'}
类型: ${config.genre || 'N/A'}
背景: ${config.setting || 'N/A'}
预估集数: ${config.episodes || 10}
单集时长: ${config.duration || 1} 分钟
视觉风格: ${config.visualStyle || 'N/A'}
${refinedReference}    ← 上面第三步拼接的精炼信息（如果有）
`;
```

6. 第五步：调用 LLM API

代码块

```
// 确定使用的模型：优先用传入的 model 参数，否则取用户配置的默认文本模型
const effectiveModel = model || getUserDefaultModel('text');

// 调用统一的 LLM 提供商管理器
const response = await llmProviderManager.generateContent(
    fullPrompt,           // 用户提示词（第四步构建的）
    effectiveModel,       // 文本模型
    {
        systemInstruction: systemInstruction  // 系统提示词（第二步替换完占位符的）
    }
);

return response;  // 直接返回 AI 生成的文本，不做任何后处理
```

`llmProviderManager.generateContent` 是统一的 LLM 调用入口：

- 支持多个提供商：Gemini、OpenAI、DeepSeek 等
- 根据用户在设置中配置的默认提供商和 API Key 自动选择
- 函数签名：`generateContent(prompt, model, options) → Promise<string>`

整个调用被包裹在 `logAPICall()` 中，用于记录 API 调用日志（模型、耗时、参数摘要等）。

7. 返回值

函数返回一个**纯 Markdown 文本字符串**。AI 按照系统提示词中的模板格式输出，大致结构：

代码块

```
# 归来
**一句话梗概**: 男主离家三年归来，发现物是人非...
**类型**: 仙侠 | **主题**: 爱情与成长 | **背景**: 古代中国 | **视觉风格**: REAL

---

## 主要人物小传

### 核心角色
* **李逍遥**: 男主 - 24岁，剑眉星目。性格：外冷内热...
* **赵灵儿**: 女主 - 22岁，柳眉杏眼。性格：温柔善良...

### 重要配角
* **苏晴**: 女主的闺蜜，性格活泼...
```

```markdown

### 其他角色
* **护士小李**: 医院护士

---

## 关键物品设定

### 核心物品
* **死神之眼徽章**: 家族传承信物，能增强持有者的超感能力...

### 辅助物品
* **医院门禁卡**: 进入特定区域的钥匙，第5集获得。

### 世界物品
* **手术刀**: 林萧的随身物品。

---

## 剧集结构规划（共 10 集，3 章）

### 第一章：觉醒篇（第1-4集）
**涉及角色**：李逍遥、赵灵儿、王院长
**关键物品**：死神之眼徽章、神秘纸条
**章节剧情**：实习医生林萧发现...神秘黑衣人出现...
- [第1集]：...
- [第2集]：...
- [第3集]：...
- [第4集]：本章高潮
**关键节点**：小高潮（第4集）

### 第二章：...
### 第三章：...
```

8. 返回值的存储

这个 Markdown 文本被存入 SCRIPT_PLANNER 节点的 `data` 中，关键字段：

存储字段	存储内容	后续谁用
`scriptOutline`	完整的 Markdown 大纲文本	②③④ 都从这个字段读取上下文
`scriptVisualStyle`	`"3D"` / `"REAL"` / `"ANIME"`	③⑤⑦ 通过递归查找读取
`scriptGenre`	`"仙侠"`	④⑤⑦ 构建提示词时读取

`scriptSetting`	`"古代中国"`	④⑤⑦ 构建提示词时读取
`scriptDuration`	`1` （分钟）	④ 计算分镜时长时读取
`scriptEpisodes`	`10`	② 分集时读取

9. 风格信息的传播机制

`scriptVisualStyle` 不是在本节点使用的。下游节点通过一个叫 `getUpstreamStyleContext` 的递归查找函数，沿着节点连线向上找到 SCRIPT_PLANNER 节点，读取风格信息。

查找逻辑（按优先级）：

代码块

```
1. 先在当前节点的直接输入节点中找 SCRIPT_PLANNER 类型的节点
   → 找到就直接读取它的 scriptVisualStyle / scriptGenre / scriptSetting

2. 如果没找到，在直接输入中找 SCRIPT_EPISODE 类型的节点
   → 然后从 SCRIPT_EPISODE 的输入中再找 SCRIPT_PLANNER
   → 找到就读取

3. 如果还是没找到，递归向上遍历所有输入节点
   → 直到找到 SCRIPT_PLANNER 为止
   → 这个处理的是间接连接的情况（比如中间隔了一个其他类型的节点）
```

找到风格后，用另一个函数 `getVisualStylePrompt` 把风格枚举转成英文提示词前缀：

代码块

```
"3D" → "Xianxia 3D animation character, semi-realistic style,
         Xianxia animation aesthetics, high precision 3D modeling,
         PBR shading with soft translucency, subsurface scattering,
         ambient occlusion, delicate and smooth skin texture,
         flowing fabric clothing, individual hair strands,
         neutral studio lighting, clear focused gaze, natural demeanor"

"REAL" → "Professional portrait photography, photorealistic human,
          cinematic photography, professional headshot,
          DSLR quality, 85mm lens, sharp focus,
          realistic skin texture, visible pores,
          natural skin imperfections, subsurface scattering,
          natural lighting, studio portrait lighting,
          softbox lighting, rim light, golden hour"

"ANIME" → "Anime character, anime style, 2D anime art,
```

```
            manga illustration style, clean linework, crisp outlines,
            manga art style, detailed illustration,
            soft lighting, rim light, vibrant colors,
            cel shading lighting, flat shading"
```

这个英文前缀会在以下场景被使用：

- 步骤 ③ 生成角色图片时 → 加在图片提示词开头
- 步骤 ⑤ 生成九宫格分镜图时 → 加在图片提示词开头
- 步骤 ⑦ 构建 Sora 视频提示词时 → 加在视频提示词开头

---

② SCRIPT_EPISODE — 剧本分集

AI 调用：文本 AI × 1 次

取 SCRIPT_PLANNER 的大纲，加上用户选择的章节号，调用 Gemini 生成分集剧本。

输入：大纲全文 + "生成第 1 集"

产出：

代码块

```
generatedEpisodes: [
  {
    title: "第1集：归来",
    content: "场景一：男主房间...\n\n李逍遥推开房门，三年未归的房间...",
    characters: ["李逍遥", "赵灵儿"],
    ...
  }
]
```

这个完整的剧本文本（`content` 字段）会传给下游两个地方：

- ③ **CHARACTER_NODE**：从文本中提取角色名
- ④ **STORYBOARD_GENERATOR**：从文本中生成分镜脚本

代码实现架构

1. 入口函数

函数名：`generateScriptEpisodes`

代码块

```
async function generateScriptEpisodes(
    outline: string,            // 剧本大纲全文（步骤①的完整 Markdown 输出）
    chapter: string,            // 用户选择的章节文本，如 "#### 第一章：觉醒篇（第1-4集）"
    splitCount: number,         // 本章节要拆分成几集，默认 3
    duration: number,           // 单集时长（分钟），从步骤①的 scriptDuration 读取
    style?: string,             // 视觉风格，"3D" / "REAL" / "ANIME"
    modificationSuggestion?: string,  // 可选，用户针对之前生成版本的修改意见
    model?: string,             // 可选，指定使用的 AI 模型
    previousEpisodes?: Array<{  // 可选，之前已生成的所有剧集（跨章节连贯性）
        title: string,
        content: string,
        characters: string,
        keyItems?: string
    }>,
    context?: { nodeId?: string; nodeType?: string }
): Promise<Array<{
    title: string,              // 如 "第1集：归来"
    content: string,            // 详细剧本内容（200-250字/分钟）
    characters: string,         // 本集涉及的角色列表
    keyItems?: string,          // 本集出现的关键物品
    continuityNote?: string     // 连贯性说明
}>>
```

2. 辅助函数：从大纲中提取角色和物品

在构建用户提示词之前，先用两个正则提取函数从大纲 Markdown 中截取角色和物品段落：

代码块

```
function extractCharactersFromOutline(outline: string): string {
    const characterSection = outline.match(/## 主要人物小传[^#]*/s);
    if (characterSection) {
        return characterSection[0].trim();
    }
    return "未找到明确的角色定义";
}

function extractItemsFromOutline(outline: string): string {
    const itemsSection = outline.match(/## 关键物品设定[^#]*/s);
    if (itemsSection) {
        return itemsSection[0].trim();
    }
    return "未找到明确的物品定义";
}
```

原理：正则 `/## 主要人物小传[^#]*/s` 匹配从 `## 主要人物小传` 开始，直到遇到下一个 `##` 标题之前的所有文本（包括换行，`s` 标志）。这样就把步骤①生成的角色小传完整段落截取出来，作为全局角色设定传给 AI。

3. 构建前序剧集摘要

如果之前已经有生成过的剧集（来自同一个或不同章节的 SCRIPT_EPISODE 节点），就把它们格式化为摘要：

代码块

```
const previousEpisodesSummary = previousEpisodes && previousEpisodes.length > 0
    ? previousEpisodes.map((ep, idx) => `
第${idx + 1}集：${ep.title}
- 涉及角色：${ep.characters}
- 关键物品：${ep.keyItems || '无'}
- 剧情摘要：${ep.content.substring(0, 200)}...
        `).join('\n')
    : '无前序剧集（这是第一批生成的剧集）';
```

关键设计：每集的剧情摘要只截取前 200 字。因为完整的剧本文本可能非常长（几百到上千字），如果全部传入，会导致提示词过长。200 字足以让 AI 了解前情提要，又不会占用太多上下文。

前序剧集的来源：在节点处理逻辑中，会收集当前画布上所有已经生成过剧集的 SCRIPT_EPISODE 节点的数据：

代码块

```
const allScriptEpisodeNodes = nodesRef.current.filter(
    n => n.type === NodeType.SCRIPT_EPISODE && n.data.generatedEpisodes &&
n.data.generatedEpisodes.length > 0
);
const previousEpisodes = allScriptEpisodeNodes.flatMap(n =>
n.data.generatedEpisodes);
```

这意味着如果用户先生成了第 1-4 集，再生成第 5-8 集，AI 会看到第 1-4 集的摘要，从而保持剧情连贯。

4. 构建用户提示词（userPrompt）

将所有输入拼接为一个完整的用户提示词：

代码块

```
const prompt = `
剧本大纲全文：
```

```
${outline}

目标章节：${chapter}
拆分集数：${splitCount}
单集时长参考：${duration} 分钟
视觉风格：${style || 'N/A'}
${modificationSuggestion ? `\n修改建议：${modificationSuggestion}` : ''}

=== 全局角色设定 ===
${globalCharacters}

=== 全局物品设定 ===
${globalItems}

=== 前序剧集摘要（用于保持连贯性）===
${previousEpisodesSummary}

连贯性要求：
1. 角色特征、说话方式必须与【全局角色设定】一致
2. 物品名称必须严格使用【全局物品设定】中的标准名称
3. 剧情应承接【前序剧集摘要】中的事件和角色状态
4. 每集的continuityNote要明确说明与剧情主线的衔接关系
`;
```

5. 系统提示词模板

系统提示词是一个固定模板，不需要占位符替换，直接作为 `systemInstruction` 传入：

代码块

```
你是一位专业的短剧分集编剧，擅长创作连贯、一致的系列剧集。
你的任务是根据提供的【剧本大纲】和【指定章节】，将该章节拆分为 N 个具体的剧集脚本。

**输入上下文：**
1. 剧本大纲 (Context) - 包含所有章节的概览
2. 目标章节 (Selected Chapter) - 当前要拆分的章节
3. 前序剧集摘要 (Previous Episodes Summary) - 之前已生成的剧集摘要，用于保持连贯性
4. 全局角色设定 (Global Characters) - 剧本大纲中定义的所有角色信息
5. 全局物品设定 (Global Items) - 剧本大纲中定义的所有关键物品信息
6. 拆分集数 (Split Count): [N]
7. 单集时长参考 (Duration Constraint)
8. 视觉风格 (Visual Style): [STYLE]
9. 修改建议 (Modification Suggestions): [如果提供] - 用户针对之前生成版本的修改意见

**连贯性和一致性要求 (CRITICAL):**

1. **角色一致性**:
```

```
    - 严格遵循【全局角色设定】中的角色外貌、性格、说话方式
    - 不要改变角色的既定特征（如：如果林霄是冷静内敛的，不要突然变得热血冲动）
    - 角色关系和互动方式要保持一致

2. **物品命名一致性**:
    - 严格使用【全局物品设定】中的标准名称
    - ❌ 错误：一会儿叫"脊骨"，一会儿叫"灵骨"
    - ✅ 正确：始终使用"脊骨"这个名称
    - 物品的特征、能力描述要保持一致

3. **剧情连贯性**:
    - 参考【前序剧集摘要】，确保时间线、事件顺序合理衔接
    - 角色的知识、状态应该承接前文（如：如果第1集主角受伤了，第2集应该体现这个状态）
    - 不要出现剧情矛盾或逻辑漏洞

4. **场景连贯性**:
    - 场景描述应该符合既定的视觉风格
    - 环境细节要保持一致（如：同一个房间的布局、装饰）

**输出要求：**
请直接输出一个 **JSON 数组**，不要包含 markdown 代码块标记（如 ```json），只输出纯 JSON 字符串。
数组中每个对象代表一集，格式如下：
[
  {
    "title": "第X集：[分集标题]",
    "content": "[详细剧本内容，包含场景描写、动作指令和对白。内容长度应符合时长限制。]",
    "characters": "[本集涉及的角色列表，必须与全局设定一致]",
    "keyItems": "[本集出现的关键物品列表，必须使用标准名称]",
    "visualStyleNote": "[针对本集的视觉风格备注]",
    "continuityNote": "[本集的连贯性说明，如承接前文哪件事、角色状态变化等]"
  },
  ...
]

**内容要求：**
1. **全中文写作**。

2. **剧本内容长度要求（CRITICAL - 必须严格遵守）**:
    - 每分钟时长需要 **200-250字** 的详细剧本内容
    - 例如：1分钟剧集 = 200-250字，2分钟剧集 = 400-500字
    - **如果对话较多，字数应相应增加**（对话+场景描述的字数密度更高）
    - 计算公式：目标字数 = 时长(分钟) × 200-250字/分钟
    - 如果内容不足，AI应该：
      * 增加更详细的场景描述（环境、光影、氛围）
      * 添加更多角色的肢体动作和表情细节
      * 扩充对话内容，增加角色互动
```

```
      * 描述角色的内心活动和情感变化
      * 加入更多感官细节（声音、气味、触感等）

3. **内容结构要求**:
   - 剧本内容 (content) 必须包含:
      * **场景描述** (Scene Action): 详细的环境描写、光影氛围、空间布局
      * **肢体动作** (Physical Actions): 角色的身体姿势、动作细节、位置移动
      * **表情细节** (Facial Expressions): 眼神、微表情、情绪变化
      * **精彩对白** (Dialogue): 符合角色性格的对话，推动剧情发展
      * **情感描写** (Emotional Depth): 内心活动、情感转变、动机暗示
   - 确保 N 个剧集能够完整覆盖所选章节的情节，并且每集结尾都要有悬念 (Cliffhanger)。
   - 场景描述应体现 [STYLE] 的视觉特点。

4. **细节扩写技巧**:
   - 不要只写"他走进房间"，要写"他推开沉重的红木门，脚步沉重地踏入昏暗的书房，皮鞋在大理石地板上发出清脆的回响"
   - 不要只写"她哭了"，要写"她的眼泪如断了线的珍珠般滑落，肩膀随着压抑的抽泣微微颤抖，双手紧紧攥着衣角，指节泛白"
   - 不要只写"房间很乱"，要写"书本散落一地，纸张如同秋风中的落叶般铺满整个房间，书架歪斜，几本书籍摇摇欲坠地挂在边缘"

5. **如果提供了修改建议，请根据建议调整剧本内容，优化情节、对白或场景描述。**

6. **在每集的continuityNote中明确说明本集与剧情主线的衔接关系。**
```

6. 调用 LLM API

代码块

```
const effectiveModel = model || getUserDefaultModel('text');

const response = await llmProviderManager.generateContent(
    prompt,             // 用户提示词（第四步构建的）
    effectiveModel,     // 文本模型
    {
        systemInstruction: SCRIPT_EPISODE_INSTRUCTION,   // 系统提示词（固定模板）
        responseMimeType: 'application/json'              // 强制 JSON 输出格式
    }
);
```

关键参数：`responseMimeType: 'application/json'` 告诉 LLM 直接输出 JSON 格式（而非 Markdown 包裹的 JSON），减少后处理复杂度。

7. 返回值解析与后处理

AI 返回的文本需要清理和解析：

代码块

```
try {
    // 清理可能的 Markdown 代码块标记（虽然设置了 responseMimeType，但有些模型仍可能输出）
    const text = response?.replace(/```json/g, '').replace(/```/g, '').trim() || "[]";
    return JSON.parse(text);
} catch (e) {
    console.error("Failed to parse script episodes JSON", e);
    throw new Error("生成剧本格式错误，请重试");
}
```

清理逻辑：

1. 移除 ````json` 和 ````` 标记
2. 去除首尾空白
3. 如果结果为空，默认返回空数组 `[]`
4. 用 `JSON.parse` 解析为对象数组
5. 解析失败时抛出明确的错误信息

8. 返回值格式示例

代码块

```
[
  {
    "title": "第1集：归来",
    "content": "场景一：李逍遥站在村口老槐树下，目光凝视着远方连绵的群山。夕阳的余晖洒在他身上，将他的影子拉得很长。三年了，他终于回来了。\n\n（李逍遥紧了紧背上的包袱，迈开步伐向村中走去）村口的小路已经长满了野草，两旁的房屋有些破旧，但他记得每一个转角、每一道门。空气中有熟悉的炊烟味道，还有远处传来的鸡鸣犬吠。\n\n（李逍遥停在自家门前，门前的大红对联已经褪色，门上的铜环生了锈）他深吸一口气，缓缓推开门。吱呀一声，门开了。屋内一切如旧，只是落满了灰尘。桌上还放着三年前他离开时的那碗面，早已干涸。\n\n（李逍遥走到桌前，伸手轻轻触碰碗的边缘，指尖微微颤抖）他闭了闭眼，记忆涌上心头——那是母亲给他做的最后一碗面。",
    "characters": "李逍遥",
    "keyItems": "包袱、褪色对联",
    "visualStyleNote": "黄昏暖色调，乡村古朴风格，REAL写实风格",
    "continuityNote": "本集为系列开篇，交代主角归乡的背景，为后续发现村子异变做铺垫"
  },
  {
    "title": "第2集：物是人非",
    "content": "...",
    "characters": "李逍遥、王婶",
    "keyItems": "半块玉佩",
```

```
      "visualStyleNote": "...",
      "continuityNote": "承接第1集李逍遥归乡，发现村民态度冷漠，引出三年间村子发生的变故"
    },
    {
      "title": "第3集：线索",
      "content": "...",
      "characters": "李逍遥、赵灵儿",
      "keyItems": "半块玉佩、神秘纸条",
      "visualStyleNote": "...",
      "continuityNote": "承接第2集发现玉佩线索，赵灵儿首次登场，两人结盟调查真相"
    }
  ]
```

9. 节点处理逻辑补充

在调用 `generateScriptEpisodes` 之前，节点处理逻辑还做了以下准备：

风格继承：如果当前 SCRIPT_EPISODE 节点没有设置视觉风格，就从上游 SCRIPT_PLANNER 节点继承：

代码块

```
let currentStyle = node.data.scriptVisualStyle;
if (!currentStyle && planner.data.scriptVisualStyle) {
    currentStyle = planner.data.scriptVisualStyle;
    handleNodeUpdate(id, { scriptVisualStyle: currentStyle });
}
```

参数来源：

参数	来源
`outline`	上游 SCRIPT_PLANNER 节点的 `scriptOutline` 字段
`chapter`	当前节点的 `selectedChapter` 字段（用户在 UI 中选择的章节）
`splitCount`	当前节点的 `episodeSplitCount` 字段，默认 3
`duration`	上游 SCRIPT_PLANNER 节点的 `scriptDuration` 字段，默认 1（分钟）
`style`	当前节点或上游节点的 `scriptVisualStyle`
`modificationSuggestion`	

	当前节点的 `episodeModificationSuggestion` 字段（用户可选填写）
`previousEpisodes`	画布上所有已生成剧集的 SCRIPT_EPISODE 节点汇总

10. 产出：子节点创建

生成成功后，为每一集创建一个 PROMPT_INPUT 类型的子节点：

代码块

```
episodes.forEach((ep, index) => {
    const newNodeId = `n-ep-${Date.now()}-${index}`;

    // 构建格式化的剧集内容
    let formattedContent = `## ${ep.title}\n\n`;
    formattedContent += `**角色**: ${ep.characters}\n`;
    if (ep.keyItems) {
        formattedContent += `**关键物品**: ${ep.keyItems}\n`;
    }
    formattedContent += `\n${ep.content}`;
    if (ep.continuityNote) {
        formattedContent += `\n\n**连贯性说明**: ${ep.continuityNote}`;
    }

    newNodes.push({
        id: newNodeId,
        type: NodeType.PROMPT_INPUT,
        title: ep.title,
        data: {
            prompt: formattedContent,  // 格式化后的完整剧本文本
            isEpisodeChild: true,      // 标记为剧集子节点
        },
        inputs: [node.id]             // 连接到当前 SCRIPT_EPISODE 节点
    });
});
```

每个子节点的 `prompt` 字段包含：标题 + 角色列表 + 关键物品 + 完整剧本内容 + 连贯性说明。

11. 存储字段映射

存储字段	存储位置	内容	下游谁用
`generatedEpisodes`	SCRIPT_EPISODE 节点	完整的剧集数组（含 title/content/characters	① 用于后续生成的前序剧集摘要；下游节点读取 `content` 字段

		/keyItems/continuityNote）	
子节点 `prompt`	新创建的 PROMPT_INPUT 子节点	格式化后的剧本文本	③ CHARACTER_NODE 从中提取角色名；④ STORYBOARD_GENERATOR 从中生成分镜脚本

核心数据流： `generateScriptEpisodes()` 返回的 JSON 数组同时写入两个地方：

1. 存入 SCRIPT_EPISODE 节点的 `generatedEpisodes`（供跨章节连贯性使用）
2. 每集拆分为独立的 PROMPT_INPUT 子节点（供下游③④节点消费）

---

③ CHARACTER_NODE — 角色设计

这个节点执行 4 个子步骤，每个角色独立处理。

③-A 提取角色名

AI 调用：文本 AI × 1 次

代码块

```
const names = await extractCharactersFromText(剧本文本);
// → ["李逍遥", "赵灵儿"]
```

AI 从剧本文本中识别出所有角色名字，存入 `extractedCharacterNames` 数组。

③-B 生成角色档案

AI 调用：文本 AI × N 次（N = 角色数）

对每个角色调用 `handleCharacterAction(GENERATE_SINGLE)`，产出：

代码块

```
generatedCharacters: [
  {
    name: "李逍遥",
    appearance: "剑眉星目，黑色长发束起...",          // 中文外貌描述
    appearancePrompt: "A young man with sharp eyebrows...", // 英文提示词（图片AI用）
    personality: "外冷内热，重情重义",
    clothing: "白色长袍，腰佩玉剑",
    roleType: "main",
```

```
      status: "SUCCESS"
    },
    {
      name: "赵灵儿",
      appearance: "清丽脱俗，柳眉杏眼...",
      appearancePrompt: "A beautiful young woman with delicate features...",
      personality: "温柔善良，内心坚强",
      clothing: "淡蓝色古装长裙",
      roleType: "main",
      status: "SUCCESS"
    }
  ]
```

`appearancePrompt` **是最关键字段**，后续所有图片生成都用它作为提示词的一部分。

③-C 生成九宫格表情图（仅主角）

AI 调用：图片 AI × N 次

用 `appearancePrompt + 风格前缀` 调用图片 AI，生成一张包含同一角色 9 种表情的图：

```
表情图提示词 = 风格前缀 + appearancePrompt + "9-grid expression sheet showing
                different emotions: happy, sad, angry, surprised, fearful,
                disgusted, neutral, determined, shy"
```

产出存入 `character.expressionSheet`（base64）。

③-D 生成三视图（仅主角）

AI 调用：图片 AI × N 次

用 `appearancePrompt + 角色档案` 生成一张包含正面、侧面、背面的图：

```
三视图提示词 = 风格前缀 + appearancePrompt + "character turnaround sheet,
                front view, side view, back view, white background"
```

产出存入 `character.threeViewSheet`（base64）。

角色参考图在整个流程中的使用时机：

参考图	被谁使用	怎么用

`threeViewSheet`	⑤ 九宫格生图	作为 `inputImages` 参数传入图片 AI
`expressionSheet`	⑤ 九宫格生图	作为 `inputImages` 的备选（无三视图时）
`threeViewSheet`	⑦-C 图片融合	拼在融合图底部/右侧
`expressionSheet`	⑦-C 图片融合	同上备选

代码实现架构

1. 节点入口逻辑

CHARACTER_NODE 节点被触发时，执行一个**两阶段**的流程：

第一阶段：提取角色名

- 检查 `node.data.extractedCharacterNames` 是否为空
- 如果为空，从上游文本中提取角色名，提取完毕后**立即返回**（不继续生成档案）
- 用户确认角色列表后，再次点击触发第二阶段

第二阶段：生成角色档案

- `extractedCharacterNames` 已有数据，遍历每个角色名
- 根据角色配置（`characterConfigs[name].method`）决定生成策略：
  - `AI_AUTO`（主角）：调用 `handleCharacterAction(GENERATE_SINGLE)` 生成完整档案
  - `SUPPORTING_ROLE`（配角）：调用 `generateSupportingCharacter()` 生成简化档案
  - `LIBRARY`（素材库）：从资产历史中直接加载
- **每个角色串行处理**（一个完成后再处理下一个）
- 生成完毕后节点状态更新为 SUCCESS

上游文本采集：

- 提取角色名时：只采集**直接输入**节点的文本（`directUpstreamTexts`）
- 生成角色档案时：采集**递归上游**所有节点的文本（`recursiveUpstreamTexts`，通过 `getUpstreamContext` 函数）

风格提示词获取：
优先级为：STYLE_PRESET 节点 > 上游递归查找 SCRIPT_PLANNER 的 `scriptVisualStyle`

2. ③-A 提取角色名

2.1 入口函数

```
代码块
1 async function extractCharactersFromText(
2     text: string,           // 上游剧本文本
3     model?: string,         // 可选，指定 AI 模型
4     context?: { nodeId?: string; nodeType?: string }
5 ): Promise<string[]>       // 返回角色名数组，如 ["李逍遥", "赵灵儿", "王婶"]
```

2.2 系统提示词

```
代码块
1 你是一位专业的选角导演。
2 你的任务是从剧本或大纲中提取所有出现的角色名称。
3 请只输出一个 JSON 字符串数组，不要包含其他内容。
4 例如: ["张三", "李四", "王五"]
```

2.3 调用方式

```
代码块
1 const response = await llmProviderManager.generateContent(
2     `提取以下剧本内容中的所有角色名: \n${text}`,
3     effectiveModel,
4     {
5         responseMimeType: 'application/json',
6         systemInstruction: CHARACTER_EXTRACTION_INSTRUCTION
7     }
8 );
```

对每个上游文本独立调用一次，然后合并去重：

```
代码块
1 const allCharacterNames: string[] = [];
2 for (const text of directUpstreamTexts) {
3     const names = await extractCharactersFromText(text);
4     allCharacterNames.push(...names);
5 }
6 const uniqueNames = Array.from(new Set(allCharacterNames.map(name =>
  name.trim()))).filter(name => name.length > 0);
```

2.4 存储与返回

提取到的角色名数组存入 `node.data.extractedCharacterNames`，同时初始化空的 `characterConfigs`。节点状态更新为 SUCCESS 后**立即返回**，等待用户再次触发。

3. ③-B 生成角色档案

角色档案生成分为**主角**和**配角**两条路径。

3.1 主角档案生成（`generateCharacterProfile`）

函数签名：

代码块

```
async function generateCharacterProfile(
    name: string,            // 角色名
    contextText: string,     // 递归上游所有文本拼接
    styleContext?: string,   // 风格提示词前缀（英文）
    customDesc?: string,     // 可选，用户自定义描述
    model?: string,          // 可选，指定 AI 模型
    apiContext?: { nodeId?: string; nodeType?: string }
): Promise<CharacterProfile>
```

用户提示词：

代码块

```
const prompt = `
Role Name: ${name}
Script Context: ${contextText}
Visual Style Context (CRITICAL): ${styleContext || "Default"}
${customDesc ? `Additional User Description: ${customDesc}` : 'Auto-complete details based on context.'}
`;
```

系统提示词（完整内容）：

代码块

```
你是一位资深的角色设计师和小说家。
你的任务是根据提供的角色名称和剧本上下文，生成极度详细的角色档案。

**输出格式要求 (JSON):**
请直接输出一个 JSON 对象，包含以下字段:
{
  "name": "角色名",
  "alias": "称谓 (同事、家人等)",
  "basicStats": "基础属性 (年龄、性别、身高、身材、发型、特征、着装)",
  "profession": "职业 (含隐藏身份)",
  "background": "生活环境、生理特征、地域标签",
  "personality": "性格 (主性格+次性格)",
```

```
  "motivation": "核心动机",
  "values": "价值观",
  "weakness": "恐惧与弱点",
  "relationships": "核心关系及影响",
  "habits": "语言风格、行为习惯、兴趣爱好",
  "appearancePrompt": "用于AI生图的详细英文提示词 (Format: [Visual Style Keywords], [Character Description], [Clothing], [Face], [Lighting]. Ensure it strictly matches the Visual Style Context provided.)"
}

**内容要求：**
1. 内容必须丰富、具体，具有画面感。
2. 必须严格遵守传入的【Visual Style Context】视觉风格设定。
3. "appearancePrompt" 字段必须包含具体的视觉风格关键词，并且描述清晰，可以直接用于文生图模型。

**视觉风格特定要求（根据 Visual Style 选择对应要求）：**

**3D动画风格（当 Visual Style 为 3D 时）：**
- 核心风格：Xianxia 3D animation character, semi-realistic style, Xianxia animation aesthetics
- 必须使用：high precision 3D modeling, PBR shading with soft translucency
- 皮肤质感：delicate and smooth skin texture (not overly realistic), subsurface scattering，追求通透柔滑质感
- 服饰细节：flowing fabric clothing, 纱质服饰的飘逸感
- 发丝细节：individual hair strands, 发丝根根分明
- 光影效果：soft ethereal lighting, cinematic rim lighting with neutral tones, ambient occlusion
- 角色气质：otherworldly gaze, elegant and natural demeanor，强化出尘气质
- 严格禁止：2D illustration, hand-drawn, anime 2D, flat shading, cel shading, toon shading, cartoon 2D, overly photorealistic, hyper-realistic skin, photorealistic rendering

**REAL真人风格（当 Visual Style 为 REAL 时）：**
- 核心风格：Photorealistic portrait, realistic human, cinematic photography, professional headshot
- 必须使用：Professional portrait photography, DSLR quality, 85mm lens, sharp focus
- 皮肤质感：Realistic skin texture, visible pores, natural skin imperfections, skin details, subsurface scattering
- 服饰细节：Realistic fabric texture, detailed clothing materials, natural fabric folds
- 发丝细节：Natural hair texture, realistic hair strands, hair volume, shiny hair
- 光影效果：Natural lighting, studio portrait lighting, softbox lighting, rim light, golden hour
```

```
- 角色气质：Natural human expression, authentic emotion, realistic gaze, professional model look
- 严格禁止：anime, cartoon, illustration, 3d render, cgi, 3d animation, painting, drawing, bad anatomy, deformed

**ANIME 2D动漫风格（当 Visual Style 为 ANIME 时）：**
- 核心风格：Anime character, anime style, 2D anime art, manga illustration style
- 必须使用：Clean linework, crisp outlines, manga art style, detailed illustration
- 皮肤质感：Smooth flat skin, cel shading, clean skin rendering, no skin texture details
- 服饰细节：Clean clothing lines, simple fabric shading, anime costume design
- 发丝细节：Stylized hair, anime hair style, sharp hair outlines, spiky hair
- 光影效果：Soft lighting, rim light, vibrant colors, cel shading lighting, flat shading
- 角色气质：Expressive anime eyes, emotional face, kawaii or cool demeanor
- 严格禁止：photorealistic, realistic, photo, 3d, cgi, live action, hyper-realistic, skin texture, pores, realistic shading

4. 如果上下文没有提供足够信息，请根据角色定位进行合理的**AI自动补全**，使其丰满。
```

返回值字段映射：

AI 返回的 JSON 被映射为 `CharacterProfile` 对象：

AI 返回字段	CharacterProfile 字段	说明
`name`	`name`	角色名
`alias`	`alias`	称谓
`basicStats`	`basicStats`	基础属性
`profession`	`profession`	职业
`personality`	`personality`	性格
`motivation`	`motivation`	核心动机
`values`	`values`	价值观
`weakness`	`weakness`	弱点
`relationships`	`relationships`	核心关系
`habits`	`habits`	习惯
`appearancePrompt`	`appearance`	

		关键字段，用于图片生成的英文提示词
全部原始数据	`rawProfileData`	保留原始 JSON

3.2 配角档案生成（`generateSupportingCharacter`）

函数签名：

代码块

```
async function generateSupportingCharacter(
    name: string,
    contextText: string,
    styleContext?: string,
    model?: string,
    apiContext?: { nodeId?: string; nodeType?: string }
): Promise<CharacterProfile>
```

用户提示词：

代码块

```
const prompt = `
Role Name: ${name}
Script Context: ${contextText}
Visual Style Context (CRITICAL): ${styleContext || "Default"}
This is a SUPPORTING CHARACTER - keep it simple and concise.
`;
```

系统提示词（完整内容）：

代码块

```
你是一位资深的角色设计师。
你的任务是为配角生成简化的角色档案。配角是故事中的次要角色，只需要基础信息即可。

**输出格式要求 (JSON):**
请直接输出一个 JSON 对象，包含以下字段:
{
  "name": "角色名",
  "basicStats": "基础属性 (年龄、性别、身高、身材、发型、特征、着装)",
  "profession": "职业",
  "introduction": "简短介绍 (1-2句话描述角色定位和在剧中的作用)",
  "appearancePrompt": "用于AI生图的详细英文提示词 (Format: [Visual Style
Keywords], [Character Description], [Clothing], [Face], [Lighting]. Ensure it
```

```
    strictly matches the Visual Style Context provided.)"
}
```

返回值字段映射：

AI 返回字段	CharacterProfile 字段	说明
`name`	`name`	角色名
`basicStats`	`basicStats`	基础属性
`profession`	`profession`	职业
`introduction`	`personality`	配角用 introduction 代替 personality
`appearancePrompt`	`appearance`	英文提示词
—	`roleType`	固定为 `'supporting'`

主角 vs 配角的核心区别：

维度	主角（AI_AUTO）	配角（SUPPORTING_ROLE）
档案详细度	11 个字段（含动机、价值观、弱点等）	5 个字段（仅基础信息）
生成表情图	需要（用户手动触发）	不需要
生成三视图	需要（用户手动触发，需先有表情图）	可以直接生成（不需要先有表情图）
处理方式	`handleCharacterAction(GENERATE_SINGLE)`	`generateSupportingCharacter()`

4. ③-C 生成九宫格表情图

4.1 触发条件

- 主角（`roleType !== 'supporting'`）
- 角色档案已生成（`state.profile` 存在）
- 用户手动点击生成，或通过 `GENERATE_ALL` 自动流程触发

4.2 提示词构建

提示词由 `promptManager.buildExpressionPrompt()` 生成，拼接方式：

```
代码块
最终提示词 = 风格前缀 + 表情图模板（中文版） + 负面提示词
```

其中：

- **风格前缀**：从步骤①传播下来的英文风格提示词（如 "Xianxia 3D animation character..."）
- **`{APPEARANCE}` 占位符**：被替换为 `profile.appearance`（即 AI 生成的 `appearancePrompt` 英文描述）
- **负面提示词**：根据风格类型不同，内容不同（见下方）

3D 风格的表情图中文提示词：

```
代码块
3D动漫风格，风格化3D渲染，PBR材质渲染，高精度3D建模，3D动漫美学。

构图：特写肖像，仅头部和肩部，专注于面部表情。

角色面部表情参考表，3×3网格布局，展示9种不同的面部表情（喜悦、愤怒、悲伤、惊讶、恐惧、厌恶、中性、思考、疲惫）。

角色面部描述：{APPEARANCE}

关键约束：
- 仅特写肖像镜头（头部和肩部）
- 无全身、无下半身、无腿部
- 专注于面部特征、表情和头部
- 纯色平背景 - 仅纯色背景（白色、浅灰色或黑色），无图案、无渐变、无环境元素
- 所有9个表情中保持一致的角色设计
- 3×3网格构图
```

REAL 风格的表情图中文提示词：

```
代码块
专业人像摄影风格，真实写实，电影级摄影质感。

构图：专业人像构图，仅头部和肩部，专注于面部表情。

角色面部表情参考表，3×3网格布局，展示9种不同的面部表情（喜悦、愤怒、悲伤、惊讶、恐惧、厌恶、中性、思考、疲惫）。

```

```
角色面部描述：{APPEARANCE}

关键约束：
- 仅特写肖像镜头（头部和肩部）
- 无全身、无下半身、无腿部
- 专注于面部特征、表情和头部
- 纯色平背景 - 仅纯色背景（白色或黑色），无图案、无渐变、无环境元素
- 所有9个表情中保持一致的角色设计
- 3×3网格构图
```

ANIME 风格的表情图中文提示词：

代码块

```
2D动漫风格，日式动漫插画，漫画艺术风格。

构图：动漫肖像构图，仅头部和肩部，专注于面部表情。

角色面部表情参考表，3×3网格布局，展示9种不同的面部表情（喜悦、愤怒、悲伤、惊讶、恐惧、厌恶、中性、思考、疲惫）。

角色面部描述：{APPEARANCE}

关键约束：
- 仅特写肖像镜头（头部和肩部）
- 无全身、无下半身、无腿部
- 专注于面部特征、表情和头部
- 纯色平背景 - 仅纯色背景（白色、浅灰色或黑色），无图案、无渐变、无环境元素
- 所有9个表情中保持一致的角色设计
- 3×3网格构图
```

负面提示词（3D 风格）：

代码块

```
nsfw, text, watermark, label, signature, bad anatomy, deformed, low quality, writing, letters, logo, interface, ui, username, website, chinese characters, chinese text, english text, korean text, japanese text, any text, any characters, any letters, numbers, symbols, subtitles, captions, title, full body, standing, legs, feet, full-length portrait, wide shot, environmental background, patterned background, gradient background, 2D illustration, hand-drawn, anime 2D, flat shading, cel shading, toon shading, cartoon 2D, paper cutout, translucent, ghostly, ethereal, glowing aura, overly photorealistic, hyper-realistic skin, photorealistic rendering
```

负面提示词（REAL 风格）：

代码块

```
nsfw, text, watermark, label, signature, bad anatomy, deformed, low quality, writing, letters, logo, interface, ui, username, website, chinese characters, chinese text, english text, korean text, japanese text, any text, any characters, any letters, numbers, symbols, subtitles, captions, title, full body, standing, legs, feet, full-length portrait, wide shot, environmental background, patterned background, gradient background, anime, cartoon, illustration, 3d render, cgi, 3d animation, painting, drawing
```

负面提示词（ANIME 风格）：

代码块

```
nsfw, text, watermark, label, signature, bad anatomy, deformed, low quality, writing, letters, logo, interface, ui, username, website, chinese characters, chinese text, english text, korean text, japanese text, any text, any characters, any letters, numbers, symbols, subtitles, captions, title, full body, standing, legs, feet, full-length portrait, wide shot, environmental background, patterned background, gradient background, photorealistic, realistic, photo, 3d, cgi, live action, hyper-realistic, skin texture, pores
```

4.3 图片 AI 调用方式

代码块

```
const exprImages = await generateImageWithFallback(
    currentPrompt,            // 中文提示词（上面构建的完整文本）
    initialModel,             // 图片模型（用户配置的优先级列表第一个）
    [],                       // inputImages 为空数组（不传入参考图）
    { aspectRatio: '1:1', count: 1 },  // 正方形比例，1张
    { nodeId, nodeType: node.type }
);
```

关键参数：

- 比例 `1:1`（正方形），因为表情图是 3×3 九宫格，需要正方形
- `inputImages` 为空——不传入任何参考图，纯靠文本提示词生成
- 使用中文提示词（不是英文）

4.4 文字检测与重试

生成后用 `detectTextInImage()` 检测图片中是否包含文字（AI 经常在九宫格中标注"喜悦""悲伤"等文字）：

代码块

```
let hasText = true;
let attempt = 0;
const MAX_ATTEMPTS = 3;

while (hasText && attempt < MAX_ATTEMPTS) {
    let currentPrompt = expressionPromptPair.zh;
    if (attempt > 0) {
        currentPrompt = currentPrompt + " NO TEXT. NO LABELS. NO LETTERS. NO CHINESE CHARACTERS. NO ENGLISH TEXT. NO WATERMARKS. CLEAN IMAGE ONLY.";
    }
    exprImages = await generateImageWithFallback(currentPrompt, ...);
    if (exprImages.length > 0) {
        hasText = await detectTextInImage(exprImages[0]);
    }
    attempt++;
}
```

最多重试 3 次，每次重试在提示词末尾追加更强的"无文字"约束。即使 3 次后仍有文字，也返回图片并输出警告。

4.5 超时保护

代码块

```
const timeoutMs = 5 * 60 * 1000;  // 5 分钟超时
const expressionSheet = await Promise.race([
    generateExpression(...),
    new Promise((_, reject) => setTimeout(() => reject(new Error('表情图生成超时')), timeoutMs))
]);
```

4.6 存储位置

生成的图片（base64）存入 `character.expressionSheet`，通过 `characterGenerationManager` 管理状态，最终同步到 `node.data.generatedCharacters`。

5. ③-D 生成三视图

5.1 触发条件

- 主角：必须先生成表情图（`state.expressionSheet` 存在），否则提示"请先生成九宫格表情图"
- 配角：可以直接生成，不需要先有表情图
- 角色档案已生成（`state.profile` 存在）

5.2 提示词构建

提示词由 `promptManager.buildThreeViewPrompt()` 生成，拼接方式：

```
代码块
1  最终提示词 = 风格前缀 + 三视图模板（中文版） + 负面提示词
```

其中：

- **风格前缀**：同表情图
- `{APPEARANCE}` **占位符**：被替换为 `profile.appearancePrompt || profile.appearance`
- `{STATS}` **占位符**：被替换为 `profile.profession`

3D 风格的三视图中文提示词：

```
代码块
1  3D动漫风格，风格化3D渲染，PBR材质渲染，高精度3D建模，3D动漫美学。
2
3  角色三视图生成任务：生成角色三视图参考表（正视图、侧视图、后视图）。
4
5  角色描述：{APPEARANCE}
6  属性：{STATS}
7
8  构图：
9  - 创建垂直布局，包含3个视图：正视图、侧视图（侧面）、后视图
10 - 全身站立姿势，中性表情
11 - 纯色平背景 - 仅纯色背景（白色、浅灰色或黑色），无图案、无渐变、无环境元素
12 - 每个视图应清晰显示指定角度的角色
13
14 关键要求：
15 1. 一致的角色设计 - 三个视图必须显示相同的角色，面部特征、发型、身体比例和服装保持一致
16 2. 无文本、无标签 - 纯图像，无"正视图"或"侧视图"文字标签
17 3. 正确的解剖结构 - 确保每个视角的正确身体比例和自然姿势
18 4. 中性表情 - 在所有视图中使用平静、中性的面部表情
19 5. 清晰对齐 - 正视图、侧视图和后视图应垂直对齐且比例一致
20
21 参考图片：使用表情图作为面部和服装细节的视觉参考。
```

REAL 风格的三视图中文提示词：

代码块

```
专业人像摄影风格，真实写实，电影级摄影质感。

角色三视图生成任务：生成角色三视图参考表（正视图、侧视图、后视图）。

角色描述：{APPEARANCE}
属性：{STATS}

构图：
- 创建垂直布局，包含3个视图：正视图、侧视图（侧面）、后视图
- 全身站立姿势，中性表情
- 纯色平背景 - 仅纯色背景（白色或黑色），无图案、无渐变、无环境元素
- 每个视图应清晰显示指定角度的角色

关键要求：
1. 一致的角色设计 - 三个视图必须显示相同的角色，面部特征、发型、身体比例和服装保持一致
2. 无文本、无标签 - 纯图像，无"正视图"或"侧视图"文字标签
3. 正确的解剖结构 - 确保每个视角的正确身体比例和自然姿势
4. 中性表情 - 在所有视图中使用平静、中性的面部表情
5. 清晰对齐 - 正视图、侧视图和后视图应垂直对齐且比例一致

参考图片：使用表情图作为面部和服装细节的视觉参考。
```

ANIME 风格的三视图中文提示词：

代码块

```
2D动漫风格，日式动漫插画，漫画艺术风格。

角色三视图生成任务：生成角色三视图参考表（正视图、侧视图、后视图）。

角色描述：{APPEARANCE}
属性：{STATS}

构图：
- 创建垂直布局，包含3个视图：正视图、侧视图（侧面）、后视图
- 全身站立姿势，中性表情
- 纯色平背景 - 仅纯色背景（白色、浅灰色或黑色），无图案、无渐变、无环境元素
- 每个视图应清晰显示指定角度的角色

关键要求：
1. 一致的角色设计 - 三个视图必须显示相同的角色，面部特征、发型、身体比例和服装保持一致
```

```
2. 无文本、无标签 - 纯图像，无"正视图"或"侧视图"文字标签
3. 正确的解剖结构 - 确保每个视角的正确身体比例和自然姿势
4. 中性表情 - 在所有视图中使用平静、中性的面部表情
5. 清晰对齐 - 正视图、侧视图和后视图应垂直对齐且比例一致

参考图片：使用表情图作为面部和服装细节的视觉参考。
```

负面提示词（三种风格共用结构，仅排除项不同）：

三视图的负面提示词相比表情图多了以下排除项：

代码块

```
patterned background, gradient background, scenery, environmental background, shadows on background
```

这是为了确保三视图背景是纯色的。

5.3 图片 AI 调用方式

代码块

```
// 使用表情图作为参考图片
const inputImages = state.expressionSheet ? [state.expressionSheet] : [];

const viewImages = await generateImageWithFallback(
    viewPrompt,                // 中文提示词
    getUserDefaultModel('image'),  // 图片模型
    inputImages,               // ← 表情图作为参考图传入
    { aspectRatio: '16:9', resolution: '1K', count: 1 },  // 横向比例，1K 分辨率
    { nodeId, nodeType: node.type }
);
```

与表情图的关键区别：

- 三视图传入 `inputImages`（表情图），表情图不传入
- 三视图比例 `16:9`（横向，因为是三个视图并排），表情图比例 `1:1`（正方形九宫格）
- 三视图有 `resolution: '1K'`，表情图没有

5.4 文字检测与重试

与表情图相同的逻辑，最多 3 次重试。重试时追加 `" NO TEXT. NO LABELS. CLEAR BACKGROUND."`。

5.5 超时保护

同表情图，5 分钟超时。

5.6 存储位置

生成的图片（base64）存入 `character.threeViewSheet`。

6. 存储字段映射

CharacterProfile 完整字段：

字段	类型	生成阶段	说明
`id`	string	系统生成	唯一标识
`name`	string	③-A	角色名
`alias`	string	③-B（仅主角）	称谓
`basicStats`	string	③-B	基础属性（年龄、性别、身高等）
`profession`	string	③-B	职业
`personality`	string	③-B	性格（主角=personality，配角=introduction）
`motivation`	string	③-B（仅主角）	核心动机
`values`	string	③-B（仅主角）	价值观
`weakness`	string	③-B（仅主角）	弱点
`relationships`	string	③-B（仅主角）	核心关系
`habits`	string	③-B（仅主角）	习惯
`appearance`	string	③-B	英文外观描述提示词（= AI 返回的 appearancePrompt）
`appearancePrompt`	string	③-B	原始 AI 生成的英文提示词
`expressionSheet`	string (base64)	③-C	九宫格表情图（仅主角）
`threeViewSheet`	string (base64)	③-D	三视图（主角+配角）
`roleType`	string	配置决定	`'main'` 或 `'supporting'`

`status`	string	各阶段更新	`'GENERATING'` / `'SUCCESS'` / `'ERROR'`

节点级存储字段：

存储字段	内容	下游谁用
`extractedCharacterNames`	角色名数组	本节点内部使用
`characterConfigs`	每个角色的生成配置（method）	本节点内部使用
`generatedCharacters`	CharacterProfile 对象数组	⑤ 读取 threeViewSheet/expressionSheet；⑦-C 读取 threeViewSheet

---

④ STORYBOARD_GENERATOR — 分镜脚本生成

④-A 生成分镜脚本

AI 调用：文本 AI × 1 次

代码块

```
const shots = await generateCinematicStoryboard(
    episodeContent,     // 第②步的剧本文本
    6,                  // storyboardCount: 6个分镜
    4,                  // storyboardDuration: 每个4秒左右
    'REAL'              // storyboardStyle
);
```

产出：`storyboardShots` 数组（`DetailedStoryboardShot[]`）

代码块

```
storyboardShots: [
  {
    shotNumber: 1,
    duration: 4,
    scene: "男主房间",
    characters: ["李逍遥", "赵灵儿"],
    shotSize: "全景",
    cameraAngle: "视平",
```

```
      cameraMovement: "轨道推拉",
      visualDescription: "男主推门走入房间，环顾四周",
      dialogue: "",
      visualEffects: "镜头从门口缓缓推进",
      audioEffects: "门轴吱呀声",
      startTime: 0,
      endTime: 4
    },
    {
      shotNumber: 2,
      duration: 3,
      scene: "男主房间",
      characters: ["赵灵儿"],
      shotSize: "中景",
      cameraAngle: "视平",
      cameraMovement: "固定",
      visualDescription: "赵灵儿惊讶地回头，手中针线掉落",
      dialogue: "",
      ...
    },
    // ... 分镜3~6 结构相同
  ]
```

这个数组是整个后续流程的"源数据"。所有下游操作都在读取这个数组的字段。

④-B 分镜预览图（可选）

AI 调用：图片 AI × M 次（并行）

对每个分镜单独生成一张预览图。这步是**可选的**，Sora 路径不依赖这些预览图。

代码实现架构

本项目存在**两套分镜生成体系**，对应两个不同的使用场景。Sora 2 路径主要使用的是**体系一**。

体系一：`generateDetailedStoryboard` — 影视级详细分镜（Sora 路径使用）

这是 Sora 2 视频生成流程使用的主流路径。它在 PROMPT_INPUT 节点（分集子节点）上触发。

1. 入口函数

代码块

```
async function generateDetailedStoryboard(
    episodeTitle: string,        // 剧集标题（从剧本内容首行提取）
    episodeContent: string,      // 剧集完整文本内容
    totalDuration: number,       // 目标总时长（秒），默认 60
```

```
    visualStyle: string,        // 视觉风格: 'REAL' | 'ANIME' | '3D', 默认 'ANIME'
    onShotGenerated?: (shot: DetailedStoryboardShot) => void,  // 单镜头回调（未使用）
    model?: string,             // 可选, 指定 AI 模型
    context?: { nodeId?: string; nodeType?: string }
): Promise<DetailedStoryboardShot[]>
```

2. 参数来源

参数	来源
`episodeTitle`	从 `node.data.prompt` 第一行提取，去掉 `#` 前缀
`episodeContent`	PROMPT_INPUT 节点的 `data.prompt`（即步骤②生成的格式化剧本文本）
`totalDuration`	上游 SCRIPT_PLANNER 节点的 `scriptDuration * 60`，默认 60 秒
`visualStyle`	上游 SCRIPT_PLANNER 节点的 `scriptVisualStyle`，默认 `'ANIME'`

上游节点查找逻辑：从当前 PROMPT_INPUT → 找到父 SCRIPT_EPISODE → 找到祖父 SCRIPT_PLANNER → 读取 `scriptDuration` 和 `scriptVisualStyle`。

3. 动态参数计算

代码块

```
const minShots = Math.floor(totalDuration / 3);        // 最低分镜数（按3秒/镜）
const recommendedShots = Math.floor(totalDuration / 2.5); // 推荐分镜数（按2.5秒/镜）
const maxShots = totalDuration;                        // 最多分镜数（按1秒/镜）
```

对照表：

总时长	最低分镜数	推荐分镜数	最多分镜数
60 秒（1分钟）	20	24	60
120 秒（2分钟）	40	48	120
180 秒（3分钟）	60	72	180

4. 系统提示词（完整内容）

代码块

```
你是一位专业的影视分镜师和摄影指导 (Storyboard Artist & DoP)。
你的任务是将提供的【剧集脚本内容】细化拆分为详细的**影视级分镜脚本**。

**输入约束: **
1. 剧集标题 (Episode Title): 提供的标题
2. 剧集内容 (Episode Content): 提供的剧本文本
3. 目标总时长 (Total Duration): [N] 秒
4. 视觉风格 (Visual Style): [STYLE]

**输出格式要求: **
必须直接输出一个 **JSON 数组**，不要包含任何 Markdown 标记（如 ```json）。
数组中每个对象代表一个分镜，格式如下:

[
  {
    "shotNumber": 1,
    "duration": 2,
    "scene": "教室 - 白天 - 靠窗最后一排",
    "characters": ["林霄"],
    "shotSize": "特写",
    "cameraAngle": "低位仰拍",
    "cameraMovement": "固定",
    "visualDescription": "(林霄坐在靠窗座位上，单手托腮，侧身望向窗外)阳光从窗外洒在林霄的侧脸上...",
    "dialogue": "无",
    "visualEffects": "浅景深，背景虚化; 暖色调光线",
    "audioEffects": "环境音 - 教室嘈杂声（低音量）"
  },
  ...
]

**拆分要求（必须严格遵守）: **

**1. 时长控制（CRITICAL - 最重要要求）**
- 每个分镜时长: 严格控制在 **1-4 秒** 之间
- 平均镜头时长: 2-3秒（保持快节奏）
- 不得出现超过4秒的长镜头
- 不得出现少于1秒的碎片化镜头

**2. 分镜数量计算（必须满足最低要求）**
根据总时长智能计算分镜数量:
- **1分钟内容（60秒）**: **至少 20 个分镜**
  - 最少: 20个分镜（平均3秒/镜）
  - 推荐: 25-30个分镜（平均2-2.4秒/镜）
```

  - 最多：60个分镜（平均1秒/镜）
- **2分钟内容（120秒）**：**至少 40 个分镜**
  - 最少：40个分镜（平均3秒/镜）
  - 推荐：50-60个分镜（平均2-2.4秒/镜）
  - 最多：120个分镜（平均1秒/镜）
- **3分钟内容（180秒）**：**至少 60 个分镜**
  - 最少：60个分镜（平均3秒/镜）
  - 推荐：75-90个分镜（平均2-2.4秒/镜）
  - 最多：180个分镜（平均1秒/镜）

3. 时间精确（强制要求）
- **所有分镜的时长总和必须等于或大于目标总时长**
- **不得低于目标总时长**（这是底线要求）
- 如果超出，允许最多超出5秒
- 例如：目标60秒，生成总时长可以在60-65秒之间

4. 时长不足的补偿策略
如果计算后发现总时长不足，必须：
- 增加更多细节镜头（如：特写角色反应、环境细节）
- 将长镜头拆分为多个短镜头
- 添加过渡镜头或转场镜头
- 补充角色表情变化的镜头
- **严禁通过增加单个镜头时长来凑时间**

5. 剧情结构智能拆分（核心要求）

关键情节/高潮场景：
- 使用更多短镜头（1-2秒），快速切换，营造紧张感

情感戏/对话场景：
- 使用中等时长镜头（2-3秒），适度推拉

环境描写/转场：
- 使用较长镜头（3-4秒），建立空间感

动作场面：
- 使用极短镜头（1秒），快速剪辑，强化动感

内容结构识别
必须识别并优先处理以下结构：
- 【场景】标记：场景切换处必须有分镜点
- 【画面】标记：画面描述重点处
- 对话：每个角色发言应有独立镜头（1-2秒）
- 动作：每个关键动作分解为1-2秒镜头
- 情绪：情绪变化点应有镜头切换

内容要求：

```

1. **专业术语**:
   - 景别：大远景、远景、全景、中景、中近景、近景、特写、大特写
   - 拍摄角度：视平、高位俯拍、低位仰拍、斜拍、越肩、鸟瞰
   - 运镜方式：固定、横移、俯仰、横摇、升降、轨道推拉、变焦推拉、正跟随、倒跟随、环绕、滑轨横移

2. **画面描述详细**:
   - **必须首先描述角色的肢体状态/身体姿势**（用括号标注）
   - 肢体状态描述应包括：身体姿势、身体状态、位置关系

3. **场景信息完整**:
   - 格式："地点 - 时间 - 具体位置"

4. **视觉效果专业**:
   - 包含景深、色调、特效、风格等信息

5. **连贯性**:
   - 分镜之间要有逻辑衔接
   - **肢体状态连贯性**：相邻分镜中角色身体姿势应保持一致

6. **对白处理**:
   - 标注角色名和对白内容
   - 如果无对白，写"无"
```

5. 用户提示词

代码块

```
const prompt = `🎯 CRITICAL TASK - 必须满足时长要求

【剧集信息】
Title: ${episodeTitle}
Content: ${episodeContent}
Duration: ${totalDuration} seconds
Style: ${visualStyle}

━━━
```

```
2️⃣ 时长总和强制要求:
   ✅ 所有分镜的duration总和 ≥ ${totalDuration} 秒
   ✅ 每个分镜时长范围: 1-4秒

3️⃣ 计算方法:
   • ${minShots} 个分镜 × 3秒 = ${minShots * 3}秒 ✅
   • ${recommendedShots} 个分镜 × 2.5秒 = ${Math.round(recommendedShots * 2.5)}秒 ✅
   • ${maxShots} 个分镜 × 1秒 = ${maxShots}秒 ✅

4️⃣ 错误示例（必须避免）:
   ❌ 只生成 ${minShots - 5} 个分镜 → 最多 ${(minShots - 5) * 4}秒 < ${totalDuration}秒 → 失败
   ✅ 生成 ${minShots} 个分镜 → ${minShots * 3}秒 ≥ ${totalDuration}秒 → 成功

━━━
```

- 缺失字段填充默认空值

4. **统计检查**（仅警告，不阻止生成）

8. 返回值数据结构

代码块

```
interface DetailedStoryboardShot {
    id: string;              // "shot-1713456789000-0"
    shotNumber: number;      // 镜头编号
    duration: number;        // 1-4 秒（已钳制）
    scene: string;           // "教室 - 白天 - 靠窗最后一排"
    characters: string[];    // ["林霄", "苏晴"]
    shotSize: string;        // 景别（8种）
    cameraAngle: string;     // 角度（6种）
    cameraMovement: string;  // 运镜（11种）
    visualDescription: string; // 视觉描述（以括号标注肢体状态开头）
    dialogue: string;        // 对白
    visualEffects: string;   // 视觉特效
    audioEffects: string;    // 音效
    startTime: number;       // 开始时间（秒）
    endTime: number;         // 结束时间（秒）
}
```

9. 存储方式

返回值被包装为 `EpisodeStoryboard` 对象存入 PROMPT_INPUT 节点：

代码块

```
const storyboard = {
    episodeTitle,                                                  // 剧集标题
    totalDuration: shots.reduce((sum, s) => sum + s.duration, 0),  // 实际总时长
    totalShots: shots.length,                                      // 分镜总数
    shots,                                                         // 分镜数组
    visualStyle                                                    // 视觉风格
};
handleNodeUpdate(id, { episodeStoryboard: storyboard });
```

10. 触发条件

1. 节点类型为 PROMPT_INPUT
2. `promptOverride === 'generate-storyboard'`
3. `node.data.prompt` 不为空且长度 >= 50
4. 该 PROMPT_INPUT 是 SCRIPT_EPISODE 的子节点（`isEpisodeChild === true`）

体系二：`generateCinematicStoryboard` — 电影分镜（STORYBOARD_GENERATOR 节点）

这是独立的 STORYBOARD_GENERATOR 节点使用的简化版分镜生成。

1. 入口函数

代码块

```
async function generateCinematicStoryboard(
    episodeScript: string,     // 上游节点传入的剧本内容
    shotCount: number,         // 分镜数量: 5-20, 默认 6
    shotDuration: number,      // 单镜时长: 2-10 秒, 默认 4
    style: string,             // 视觉风格: 'REAL' | 'ANIME' | '3D', 默认 'REAL'
    model?: string,
    context?: { nodeId?: string; nodeType?: string }
): Promise<StoryboardShot[]>
```

2. 系统提示词

代码块

```
你是一位世界级的电影导演和摄影指导 (Director of Photography)。
你的任务是根据提供的【剧集脚本】，将其拆解为一系列专业的**电影分镜头 (Cinematic Shots)**。

**输入约束: **
1. 剧集内容 (Episode Content): 提供的文本。
2. 拆解数量 (Shot Count): [N] 个镜头。
3. 镜头时长 (Shot Duration): [T] 秒。
4. 视觉风格 (Visual Style): [STYLE]

**输出格式要求: **
必须直接输出一个 **JSON 数组**, 不要包含任何 Markdown 标记。
数组中每个对象必须严格包含以下字段（全部为 String 类型）:

[
  {
    "subject": "主体: [详细描述人物外貌、动作、情绪]",
    "scene": "场景: [时间、地点、光影、氛围]",
    "camera": "镜头语言: [景别、角度、运镜方式、焦点]",
    "lighting": "光影: [光源性质、光比、色调]",
    "dynamics": "动态与特效: [环境动态、物理特效]",
    "audio": "声音: [人声、音效、BGM]",
    "style": "风格与质感: [画面风格、分辨率、胶片感]",
```

```
    "negative": "负面约束：[禁止出现的内容]"
  },
  ...
]

**内容创作要求：**
1. **视觉语言专业化**：使用专业的电影术语。
2. **画面感极强**：描述必须极其具体。
3. **连贯性**：镜头之间要有逻辑衔接。
4. **情感传递**：通过光影和构图强化角色的情绪。
5. **风格一致性**：确保所有镜头的描述符合指定的 [STYLE]。
```

3. 用户提示词

代码块

```
const prompt = `
Episode Script: ${episodeScript}
Shot Count: ${shotCount}
Shot Duration: ${shotDuration}s
Visual Style: ${style}
`;
```

4. 返回值数据结构

代码块

```
interface StoryboardShot {
    id: string;           // "shot-1713456789000-0"
    subject: string;      // "主体：..."
    scene: string;        // "场景：..."
    camera: string;       // "镜头语言：..."
    lighting: string;     // "光影：..."
    dynamics: string;     // "动态与特效：..."
    audio: string;        // "声音：..."
    style: string;        // "风格与质感：..."
    negative: string;     // "负面约束：..."
    imageUrl?: string;    // 生成的预览图 Base64（后续填充）
    duration?: number;    // 固定为 shotDuration 参数值
}
```

与体系一的区别：体系二的每个镜头时长相同（固定为 `shotDuration`），且字段结构不同（用 subject/camera/lighting 等代替 shotSize/cameraAngle/cameraMovement）。

④-B 分镜预览图

分镜预览图在 STORYBOARD_GENERATOR 节点中**自动触发**（非可选）。生成成功后为每个镜头并发生成预览图：

代码块

```
// 风格前缀映射
const stylePrompt = style === 'ANIME'
    ? 'Anime style, Japanese animation, Studio Ghibli style, 2D, Cel shaded, vibrant colors.'
    : style === '3D'
    ? 'Xianxia 3D animation character, semi-realistic style, Xianxia animation aesthetics, high precision 3D modeling, PBR shading with soft translucency, subsurface scattering, ambient occlusion, delicate and smooth skin texture (not overly realistic), flowing fabric clothing, individual hair strands, soft ethereal lighting, cinematic rim lighting with cool blue tones, otherworldly gaze, elegant and cold demeanor, 3D animation quality, vibrant colors.'
    : 'Cinematic Movie Still, Photorealistic, 8k, Live Action, highly detailed.';

// 单镜头提示词
const visualPrompt = `
${stylePrompt}
Subject: ${shot.subject}.
Scene: ${shot.scene}.
Camera: ${shot.camera}.
Lighting: ${shot.lighting}.
Style: ${shot.style}.
Negative: ${shot.negative}.
`;

// 并行生成所有预览图
await Promise.all(updatedShots.map((_, i) => processShotImage(i)));
```

每生成一张图片，立即更新对应镜头的 `imageUrl` 字段。单个镜头失败只输出警告，不中断流程。

存储字段映射

存储字段	存储位置	内容	下游谁用
`episodeStoryboard`	PROMPT_INPUT 节点	包含 shots 数组、总时长、总镜头数	⑤ 读取 shots 生成九宫格图
`storyboardShots`	STORYBOARD_GENERATOR 节点	体系二的分镜数组	旧流程使用

⑤ STORYBOARD_IMAGE — 分镜九宫格图生成

5.1 计算分页

```
代码块
1   九宫格模式：每页 9 格
2   总分镜数：6
3   总页数 = Math.ceil(6 / 9) = 1 页
```

6 个分镜只需要 1 页，所以只调 **1 次图片 AI**。

如果是 14 个分镜，就需要 2 页，每页独立并行生成。

5.2 从 storyboardShots 取数据

```
代码块
1   第 0 页: storyboardShots.slice(0, 9)  → 取出分镜 1~6 的数据
```

对每个分镜，`buildDetailedShotPrompt` 函数拼接英文描述：

```
代码块
1   分镜1 的英文描述:
2     visualDescription: "Character A pushes open the door and walks into the room"
3     shotSize 映射:     "full shot, entire body visible, head to toe"
4     cameraAngle 映射:  "eye-level angle, neutral and natural perspective"
5     cameraMovement 映射: "tracking shot"
6     scene:             "environment: Character A's room"
7
8     拼接结果: "Character A pushes open the door and walks into the room.
9               full shot, entire body visible, head to toe. tracking shot.
10              environment: Character A's room"
```

注意：中文名被 `sanitizeChineseNames` 替换了。"李逍遥" → "Character A"，"赵灵儿" → "Character B"。因为图片 AI 对英文理解更好。

5.3 同场景一致性约束

代码遍历所有分镜的 `scene` 字段，发现分镜 1、2、3、4、5、6 都是"男主房间"，就自动生成一段约束文本：

代码块

```
SCENE CONSISTENCY REQUIREMENTS:
CRITICAL: Panels belonging to the same scene MUST maintain perfect visual consistency:
- Location 1 (Panels 1, 2, 3, 4, 5, 6): 男主推门走入房间...

For each scene above:
- Environment style, architecture, and props must be IDENTICAL
- Lighting quality, color temperature must be CONSISTENT
- Background elements, textures must be the same
```

这确保同一场景的不同面板在九宫格图中看起来像同一个地方。

5.4 角色参考图的传入

从上游 CHARACTER_NODE 拿到角色的 `threeViewSheet` 或 `expressionSheet`（base64），作为图片 AI 的 `inputImages` 参数：

代码块

```
generateImageWithFallback(
    gridPrompt,                // 超长文本提示词
    primaryImageModel,         // Gemini 图片模型
    characterReferenceImages,  // ← [三视图1(base64), 三视图2(base64)]
    { aspectRatio, resolution, count }
)
```

图片 AI 同时看到文本提示词和角色参考图，生成九宫格时角色就会长得像参考图。

5.5 完整的提示词长什么样

代码块

```
Create a professional cinematic storyboard 3x3 grid layout at 1K resolution.

OVERALL IMAGE SPECS:
- Output Aspect Ratio: 48:27 (landscape)
- Grid Layout: 9 panels arranged in 3x3 formation (3 columns × 3 rows)
- Each panel: 16:9 aspect ratio (landscape/horizontal)
- Panel borders: EXACTLY 4 pixels wide black lines

QUALITY STANDARDS:
- Professional film industry storyboard quality
- ALL characters must look identical across all panels
- Same color palette, same art style, same lighting quality throughout

CRITICAL NEGATIVE CONSTRAINTS:
```

```
- NO text, NO speech bubbles, NO dialogue boxes
- NO subtitles, NO captions, NO watermarks

ART STYLE: Professional portrait photography, photorealistic human...

CHARACTER CONSISTENCY:
⚠️ MANDATORY: Character A = reference image 1, Character B = reference image 2
- Face: SAME facial features across all panels
- Hair: IDENTICAL hairstyle, hair color
- Clothing: EXACT SAME clothes, accessories

SCENE CONSISTENCY REQUIREMENTS:
- Location 1 (Panels 1, 2, 3, 4, 5, 6): ...

PANEL BREAKDOWN:
---
16:9 landscape - Character A pushes open the door... full shot... tracking shot...
---
16:9 landscape - Character B turns around in surprise... medium shot... static...
---
...
```

5.6 产出

代码块

```
storyboardGridImages: [
  "..."   // 一张九宫格大图
]
```

注意：`storyboardShots` 数组没有变化，还是原来的 6 个元素。九宫格图只是把这个数组的视觉化呈现。**

代码实现架构

1. 入口逻辑

当 `node.type === STORYBOARD_IMAGE` 时进入。完整流程：

1. 判断是否为面板/页面重新生成请求（检查 `storyboardRegeneratePanel`）
2. 获取或解析分镜描述数据（`extractedShots`）
3. 获取网格配置（`gridType`、`panelOrientation`、分辨率）

4. 计算整体图片宽高比和输出尺寸
5. 获取上游视觉风格上下文
6. 从上游 CHARACTER_NODE 提取角色参考图
7. 执行内部辅助函数：`sanitizeChineseNames`、`buildDetailedShotPrompt`、生成场景一致性文本
8. 逐页生成九宫格图片（并行，带 5 分钟超时）
9. 保存结果到节点数据

2. 分镜数据提取

优先级：

1. `node.data.storyboardShots`（非重新生成时直接使用已有数据）
2. 上游 PROMPT_INPUT 节点的 `episodeStoryboard.shots`（体系一的详细分镜）
3. JSON 解析回退：先 `JSON.parse` 整体 → 再正则提取 `shots` 字段 → 再按文本行解析
4. 文本回退：匹配 `数字+分隔符` 格式，或按换行拆分过滤长度 > 10 的行

3. 分页计算

代码块

```
const gridConfig = getGridConfig(gridType);
const shotsPerGrid = gridConfig.shotsPerGrid;
const numberOfPages = Math.ceil(extractedShots.length / shotsPerGrid);
```

gridType	shotsPerGrid	gridLayout	cols	rows
`'4'`	4	`'2x2'`	2	2
`'9'`	9	`'3x3'`	3	3
`'16'`	16	`'4x4'`	4	4

默认返回 `'9'`（3×3 九宫格）。

4. 宽高比与尺寸计算

面板方向： `'16:9'`（横屏）或 `'9:16'`（竖屏），默认 `'16:9'`。

整体宽高比： 每面板单位尺寸 × 网格列行数 → 求最大公约数（GCD）简化 → 映射到 API 最近支持的比例：`['1:1', '4:3', '3:4', '16:9', '9:16', '21:9', '9:21']`

输出尺寸：

分辨率	目标像素
`'1k'`	1024 × 1024
`'2k'`	2048 × 2048
`'4k'`	4096 × 4096

根据宽高比计算实际宽高，取整为 8 的倍数。单面板尺寸 = 总尺寸 / 列或行数。

5. `sanitizeChineseNames` 函数

将中文角色名替换为英文代码，避免 AI 生成图片时出现中文字符。

代码块

```
// 构建映射表
const characterNameMap: Map<string, string> = new Map();
const labels = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
// 第1个角色 → "Character A"，第2个 → "Character B"，超过26个用数字索引

const sanitizeChineseNames = (text: string): string => {
    let result = text;
    for (const [cnName, enCode] of characterNameMap) {
        result = result.replace(new RegExp(cnName, 'g'), enCode);
    }
    return result;
};
```

6. `buildDetailedShotPrompt` 函数

为每个分镜面板构建英文提示词片段，由以下 4 部分用 `.` 连接：

1. `visualDescription`（经过 `sanitizeChineseNames` 处理后的视觉描述）
2. `shotSize` 对应的英文景别描述
3. `cameraAngle` 对应的英文角度描述
4. `scene` 场景上下文（格式：`environment: {场景名}`）

景别映射表：

中文	英文
`'大远景'`	`'extreme long shot, vast environment, figures small like ants'`

`'远景'`	`'long shot, small figure visible, action and environment'`
`'全景'`	`'full shot, entire body visible, head to toe'`
`'中景'`	`'medium shot, waist-up composition, social distance'`
`'中近景'`	`'medium close-up shot, chest-up, focus on emotion'`
`'近景'`	`'close shot, neck and above, intimate examination'`
`'特写'`	`'close-up shot, face only, soul window, intense impact'`
`'大特写'`	`'extreme close-up shot, partial detail, microscopic view'`

拍摄角度映射表：

中文	英文
`'视平'`	`'eye-level angle, neutral and natural perspective'`
`'高位俯拍'`	`'high angle shot, looking down at subject, makes them appear vulnerable'`
`'低位仰拍'`	`'low angle shot, looking up at subject, makes them appear powerful'`
`'斜拍'`	`'dutch angle, tilted horizon, creates psychological unease'`
`'越肩'`	`'over the shoulder shot, emphasizes relationship and space'`
`'鸟瞰'`	`"bird's eye view, top-down 90-degree, god-like perspective"`

注意： `cameraMovement` （运镜方式）字段在 `buildDetailedShotPrompt` 中**未被映射使用**，只在后续步骤 ⑦ 的 Sora 提示词中使用。

7. 场景一致性约束文本

遍历当前页所有分镜的 `scene` 字段，按场景名分组。生成的文本模板：

```
代码块

SCENE CONSISTENCY REQUIREMENTS:
CRITICAL: Panels belonging to the same scene MUST maintain perfect visual consistency:
- Location 1 (Panels {面板号列表}): {合并描述摘要}

For each scene above:
- Environment style, architecture, and props must be IDENTICAL across all panels of that scene
- Lighting quality, color temperature, and shadow direction must be CONSISTENT within the same scene
- Atmosphere, mood, and environmental effects must match across panels of the same scene
- Background elements, textures, and materials must be the same for the same scene
- Time of day and weather conditions must be consistent within each scene

This ensures visual continuity - multiple panels showing the same scene should look like different camera angles of the SAME location, not different places.
```

场景名被替换为 `Location N` （N 从 1 递增），描述合并后截断至 150 字符。

8. 角色参考图传入

从上游 CHARACTER_NODE 节点的 `generatedCharacters` 中提取：

- 优先取 `threeViewSheet` （三视图）
- 其次取 `expressionSheet` （表情图）
- 放入 `characterReferenceImages` 数组
- 作为 `generateImageWithFallback` 的 `inputs` 参数传入

当有参考图时，提示词中追加 CHARACTER CONSISTENCY 段落：

```
代码块

CHARACTER CONSISTENCY (CRITICAL):
⚠️ MANDATORY: You MUST use the provided character reference images as the ONLY source of truth for character appearance.

Character A = reference image 1
Character B = reference image 2
```

```
...

REQUIREMENTS:
- ALL characters in EVERY panel must look EXACTLY THE SAME as in the reference images
- Face: SAME facial features, eye shape, nose, mouth, skin tone, expression style
- Hair: IDENTICAL hairstyle, hair color, hair texture, hair length
- Body: SAME body proportions, height, build, posture
- Clothing: EXACT SAME clothes, accessories, shoes, colors, fabrics
- Skin: IDENTICAL skin texture, skin tone, skin quality
- ZERO tolerance for character appearance changes across panels

This is NON-NEGOTIABLE: Character consistency across all panels is mandatory.
```

9. 完整的九宫格提示词模板

代码块

```
Create a professional cinematic storyboard {gridLayout} grid layout at {resolution} resolution.

OVERALL IMAGE SPECS:
- Output Aspect Ratio: {imageAspectRatio} ({orientation})
- Grid Layout: {shotsPerGrid} panels arranged in {gridLayout} formation ({cols} columns × {rows} rows)
- Each panel: {panelOrientation} aspect ratio
- Panel borders: EXACTLY 4 pixels wide black lines
- CRITICAL: All panel borders must be PERFECTLY UNIFORM

QUALITY STANDARDS:
- Professional film industry storyboard quality
- **{resolution} HD resolution ({baseWidth} pixels wide base)**
- High-detail illustration with sharp focus
- CRITICAL: Maintain 100% visual style consistency across ALL panels
- ALL characters must look identical across all panels

CRITICAL NEGATIVE CONSTRAINTS (MUST FOLLOW):
- NO text, NO speech bubbles, NO dialogue boxes
- NO subtitles, NO captions, NO watermarks
- NO letters, NO numbers, NO typography, NO panel numbers
- NO markings or labels of any kind

{如果 stylePrefix 非空: ART STYLE: {stylePrefix}}

{如果有角色参考图: CHARACTER CONSISTENCY 段落（见第 8 节）}
```

```

{如果有同场景面板: SCENE CONSISTENCY REQUIREMENTS 段落（见第 7 节）}

PANEL BREAKDOWN (each panel MUST be visually distinct, separated by ---):
---
{panelOrientation} - {buildDetailedShotPrompt(shot0)}
---
{panelOrientation} - {buildDetailedShotPrompt(shot1)}
...
（共 shotsPerGrid 个面板，超出实际分镜数的用 "(empty panel - storyboard end)" 标记）

COMPOSITION REQUIREMENTS:
- Each panel MUST depict a DIFFERENT scene/angle/moment
- NO repetition of content between panels
- Maintain narrative flow across the {gridLayout} grid

ABSOLUTE RULE - NO TEXT IN IMAGE:
The generated image must contain ZERO text, ZERO letters, ZERO numbers, ZERO labels.
The ONLY exception: in-scene text on physical objects (e.g. signboard, book cover).
```

10. 图片 AI 调用

代码块

```
const imgs = await generateImageWithFallback(
    gridPrompt,                    // 完整提示词
    primaryImageModel,             // 用户配置的图片模型优先级第一个
    characterReferenceImages,       // 角色参考图数组
    {
        aspectRatio: imageAspectRatio,  // 整体比例
        resolution: resolution.toUpperCase(),  // '1K' / '2K' / '4K'
        count: 1
    },
    { nodeId, nodeType }
);
```

降级机制：同一模型重试 2 次后降级到下一个模型。**超时**：每页 5 分钟。**并行**：所有页用 `Promise.all` 并行。

11. 存储字段映射

字段	内容	下游谁用

`storyboardGridImages`	所有页面图片 URL 数组	⑥ 读取并切割
`storyboardGridImage`	第一张图片 URL（向后兼容）	旧流程使用
`storyboardShots`	提取的分镜数据数组	⑥ 读取并关联到切割面板

---

⑥ STORYBOARD_SPLITTER — 分镜图拆解

不调用任何 AI，纯 Canvas 操作。

6.1 切割原理

```
代码块
九宫格: 3列 × 3行
原始大图尺寸: 比如 1536×864

每格宽度 = 1536 / 3 = 512
每格高度 = 864 / 3 = 288
```

用 `ctx.drawImage(img, x, y, w, h, 0, 0, w, h)` 从大图里裁出每一格：

```
代码块
panelIndex 0: row=0, col=0 → 裁 (0, 0, 512, 288)       → 分镜1
panelIndex 1: row=0, col=1 → 裁 (512, 0, 512, 288)      → 分镜2
panelIndex 2: row=0, col=2 → 裁 (1024, 0, 512, 288)     → 分镜3
panelIndex 3: row=1, col=0 → 裁 (0, 288, 512, 288)      → 分镜4
panelIndex 4: row=1, col=1 → 裁 (512, 288, 512, 288)     → 分镜5
panelIndex 5: row=1, col=2 → 裁 (1024, 288, 512, 288)    → 分镜6
panelIndex 6: row=2, col=0 → 裁 (0, 576, 512, 288)       → 空白
panelIndex 7: row=2, col=1 → 裁 (512, 576, 512, 288)      → 空白
panelIndex 8: row=2, col=2 → 裁 (1024, 576, 512, 288)     → 空白
```

对应关系纯靠数组下标：九宫格的第 i 格 = storyboardShots[i]。

6.2 数据合并

切出的图片（base64）和 storyboardShots 的描述数据合并成 `SplitStoryboardShot`：

```
代码块
splitShot = {
    // 新增字段
```

```
    id: "split-node1-0-0",
    splitImage: canvas.toDataURL(),    // ← 切出的小图
    sourceNodeId: "node1",             // ← 来自哪个 STORYBOARD_IMAGE 节点
    sourcePage: 0,                     // ← 来自第几页
    panelIndex: 0,                     // ← 九宫格第几格

    // 从 storyboardShots[0] 原样继承
    scene: "男主房间",
    characters: ["李逍遥", "赵灵儿"],
    shotSize: "全景",
    cameraAngle: "视平",
    cameraMovement: "轨道推拉",
    visualDescription: "男主推门走入房间，环顾四周",
    dialogue: "",
    visualEffects: "镜头从门口缓缓推进",
    audioEffects: "门轴吱呀声",
    duration: 4,
}
```

6.3 产出

代码块

```
splitShots: [
  { panelIndex: 0, splitImage: "...", scene: "男主房间", ... }, // 分镜1
  { panelIndex: 1, splitImage: "...", scene: "男主房间", ... }, // 分镜2
  { panelIndex: 2, splitImage: "...", scene: "男主房间", ... }, // 分镜3
  { panelIndex: 3, splitImage: "...", scene: "男主房间", ... }, // 分镜4
  { panelIndex: 4, splitImage: "...", scene: "男主房间", ... }, // 分镜5
  { panelIndex: 5, splitImage: "...", scene: "男主房间", ... }, // 分镜6
  { panelIndex: 6, splitImage: "...", scene: "", ... }, // 空白
  { panelIndex: 7, splitImage: "...", scene: "", ... }, // 空白
  { panelIndex: 8, splitImage: "...", scene: "", ... }, // 空白
]
```

到这一步，每个分镜同时拥有"图片"（splitImage）和"描述数据"（所有字段），绑在一起了。

代码实现架构

不调用任何 AI，纯 Canvas 操作。

1. `splitStoryboardImage` — 单张九宫格切割

函数签名：

```
代码块
1  async function splitStoryboardImage(
2      imageUrl: string,           // 图片 URL（data URL 或 HTTP URL）
3      gridType: '9' | '6',        // 网格类型
4      shotsData: any[],           // 当前页的分镜描述数据数组
5      sourceNodeId: string,       // 来源 STORYBOARD_IMAGE 节点 ID
6      sourcePage: number          // 来源页码（0-based）
7  ): Promise<SplitStoryboardShot[]>
```

执行流程：

第一步：加载图片

创建 `Image` 对象，如果 URL 以 `http` 开头则设置 `crossOrigin = 'anonymous'` 处理 CORS。

第二步：计算网格布局

gridType	cols	rows	panelCount
`'9'`	3	3	9
`'6'`	3	2	6

第三步：计算每格尺寸

```
代码块
1  const panelWidth = img.width / cols;
2  const panelHeight = img.height / rows;
```

第四步：逐格切割

```
代码块
1  for (let i = 0; i < panelCount; i++) {
2      const row = Math.floor(i / cols);    // 整除得到行号
3      const col = i % cols;                // 取余得到列号
```

```
    const x = col * panelWidth;          // 原图 X 起始坐标
    const y = row * panelHeight;         // 原图 Y 起始坐标

    const canvas = document.createElement('canvas');
    canvas.width = panelWidth;
    canvas.height = panelHeight;
    const ctx = canvas.getContext('2d');
    ctx.drawImage(
        img,
        x, y, panelWidth, panelHeight,     // 源: 起始坐标 + 裁切尺寸
        0, 0, panelWidth, panelHeight      // 目标: canvas 起始 + 绘制尺寸
    );
    const splitImage = canvas.toDataURL('image/png', 0.95);
}
```

第五步：数据合并

切割后的小图与分镜描述数据合并为 `SplitStoryboardShot`：

代码块

```
const splitShot = {
    id: `split-${sourceNodeId}-${sourcePage}-${i}`,
    shotNumber: i + 1,
    sourceNodeId,
    sourcePage,
    panelIndex: i,
    splitImage,                    // ← 切出的 Base64 小图
    // 以下字段从 shotsData[i] 原样继承
    scene: shotData.scene || '',
    characters: shotData.characters || [],
    shotSize: shotData.shotSize || '',
    cameraAngle: shotData.cameraAngle || '',
    cameraMovement: shotData.cameraMovement || '',
    visualDescription: shotData.visualDescription || shotData.description || '',
    dialogue: shotData.dialogue || '',
    visualEffects: shotData.visualEffects || '',
    audioEffects: shotData.audioEffects || '',
    startTime: shotData.startTime || 0,
    endTime: shotData.endTime || 0,
    duration: shotData.duration || 3
};
```

2. `splitMultipleStoryboardImages` — 多页批量处理

函数签名：

代码块

```
async function splitMultipleStoryboardImages(
    nodes: Array<{
        id: string;
        data: {
            storyboardGridImages?: string[];
            storyboardGridType?: string;
            storyboardShots?: any[];
        };
    }>,
    onProgress?: (current: number, total: number, currentNode: string) => void
): Promise<SplitStoryboardShot[]>
```

执行流程：

1. 计算总处理数（所有节点的 `storyboardGridImages.length` 之和）
2. 逐节点 → 逐页处理：
   - 每页分镜数 = `gridType === '9' ? 9 : 6`
   - 当前页数据 = `allShots.slice(pageIndex * shotsPerGrid, (pageIndex + 1) * shotsPerGrid)`
   - 调用 `splitStoryboardImage` 切割
3. **容错**：单页失败时 `catch` 并 `console.error`，不中断整体流程
4. 触发进度回调
5. 返回所有切割后的 `SplitStoryboardShot` 数组

3. `SplitStoryboardShot` 完整字段

代码块

```
interface SplitStoryboardShot {
    // 标识信息
    id: string;                    // "split-{sourceNodeId}-{sourcePage}-{panelIndex}"
    shotNumber: number;            // 面板编号（1-based）
    sourceNodeId: string;          // 来源节点 ID
    sourcePage: number;            // 来源页码（0-based）
    panelIndex: number;            // 面板索引（0-based）
    splitImage: string;            // Base64 PNG 图片

    // 分镜描述数据（从 DetailedStoryboardShot 继承）
```

```
    scene: string;
    characters: string[];
    shotSize: string;
    cameraAngle: string;
    cameraMovement: string;
    visualDescription: string;
    dialogue: string;
    visualEffects: string;
    audioEffects: string;
    startTime: number;
    endTime: number;
    duration: number;
}
```

4. 存储字段映射

存储字段	内容	下游谁用
`splitShots`	SplitStoryboardShot 数组	⑦ 读取并按时长分组

---

⑦ SORA_VIDEO_GENERATOR — Sora 2 视频生成

⑦-A 按时长自动分组

不调用 AI，纯逻辑计算。

从 STORYBOARD_SPLITTER 节点拿到所有 splitShots，按顺序逐个塞进 10 秒的"桶"里：

代码块

```
遍历 splitShots:

当前桶 = 空桶（0秒）

遇到分镜1(4秒): 桶 = 4秒 → 没超10秒 → 塞进去
遇到分镜2(3秒): 桶 = 7秒 → 没超10秒 → 塞进去
遇到分镜3(5秒): 桶 = 12秒 → 超了！封存桶为"任务组1"，开新桶
                    新桶 = 5秒 → 塞进去
遇到分镜4(4秒): 桶 = 9秒 → 没超10秒 → 塞进去
遇到分镜5(3秒): 桶 = 12秒 → 超了！封存桶为"任务组2"，开新桶
                    新桶 = 3秒 → 塞进去
遇到分镜6(3秒): 桶 = 6秒 → 没超10秒 → 塞进去
循环结束，封存为"任务组3"
```

分组结果（剧情连续，按顺序切）：

代码块

```
taskGroups: [
  {
    taskNumber: 1, totalDuration: 7,
    splitShots: [分镜1对象, 分镜2对象],       // ← 带图+数据的完整对象
    shotIds: ["split-node1-0-0", "split-node1-0-1"]
  },
  {
    taskNumber: 2, totalDuration: 9,
    splitShots: [分镜3对象, 分镜4对象],
    shotIds: ["split-node1-0-2", "split-node1-0-3"]
  },
  {
    taskNumber: 3, totalDuration: 6,
    splitShots: [分镜5对象, 分镜6对象],
    shotIds: ["split-node1-0-4", "split-node1-0-5"]
  }
]
```

空白面板（panelIndex 6/7/8）因为 duration=3 且 scene=""，也会被分配到任务组里。但它的描述是空的，对 Sora 生成影响不大。

⑦-B 为每个任务组构建提示词

AI 调用：文本 AI × 3 次（每个任务组 1 次）

对每个任务组，把它的 `splitShots` 传给 `sora2Builder.build()`：

代码块

```
tg.soraPrompt = await sora2Builder.build(tg.splitShots, {
    includeBlackScreen: true,
    blackScreenDuration: 0.5
});
```

AI 从 splitShots 的字段中提取信息，生成多镜头格式提示词：

任务组 1 的提示词：

代码块

```
Shot 1:
duration: 0.5s
```

```
Scene: 纯黑空镜，无任何视觉内容

Shot 2:
duration: 4.0s
Scene: 全景，视平，轨道推拉，男主推门走入房间，环顾四周，[门轴吱呀声]

Shot 3:
duration: 3.0s
Scene: 中景，视平，固定，女主惊讶地回头，手中针线掉落
```

提示词内容来源：

- `duration` ← `splitShots[i].duration`
- 景别 ← `splitShots[i].shotSize`（如"全景"）
- 角度 ← `splitShots[i].cameraAngle`（如"视平"）
- 运镜 ← `splitShots[i].cameraMovement`（如"轨道推拉"）
- 描述 ← `splitShots[i].visualDescription`（如"男主推门走入房间"）
- 对白 ← `splitShots[i].dialogue`（如有）
- 音效 ← `splitShots[i].audioEffects`（如有）

如果 AI 生成失败，走降级函数 `buildSoraStoryPrompt`，不做 AI 加工，直接拼接基本格式。

⑦-C 图片融合（用户可选）

不调用 AI，Canvas 拼图。

对每个任务组，用 Canvas 把它的 splitShots 里的 `splitImage` 拼成一张参考图。

任务组 1 的融合过程：

代码块

```
1. 加载分镜1.splitImage 和 分镜2.splitImage
2. 计算网格布局: 2张图 → 1行2列
3. 在 Canvas 上绘制:
   ┌──────────┬──────────┐
   │ ①        │ ②        │   ← 每张图左上角画编号（白字黑底）
   │ 分镜1图  │ 分镜2图  │
   └──────────┴──────────┘
4. 如果有角色三视图，追加到底部:
   ┌──────────┬──────────┐
   │ ①        │ ②        │
   │ 分镜1图  │ 分镜2图  │
   ├──────────┴──────────┤
   │ 角色三视图           │
```

```
    │ [正面]  [侧面]  [背面] │
    └──────────────────────────┘
5. 顶部加标题行: "分镜融合图 - 共 2 个镜头"
6. canvas.toDataURL() → base64
7. 上传到阿里云 OSS → 得到 https://xxx.aliyuncs.com/sora-reference-xxx.png
8. URL 存入 taskGroup.referenceImage
```

为什么要上传 OSS? 因为 Sora 2 API 只接受公网可访问的图片 URL，不接受 base64。代码里有检查:

代码块

```
if (referenceImageUrl && referenceImageUrl.startsWith('data:')) {
    // base64 格式，必须上传 OSS 转成 URL
    referenceImageUrl = await uploadFileToOSS(base64Data, fileName, ossConfig);
}
```

如果用户没配置 OSS，融合图就传不上去，Sora 只能靠纯文本提示词生成（无参考图）。

每个任务组的参考图是独立的，只包含该组的分镜小图 + 角色参考。

⑦-D 提交 Sora 2 视频生成

AI 调用：视频 AI × 3 次（串行，一个做完才做下一个）

代码块

```
for (taskGroup of taskGroups) {
    await generateSoraVideo(taskGroup);
}
```

每次生成的完整流程:

代码块

```
1. 检查参考图格式
   referenceImage = taskGroup.referenceImage
   如果是 base64 → 上传 OSS → 转成 URL
   如果是空 → 警告"无参考图"

2. 提交任务
   POST → provider.submitTask({
       prompt: taskGroup.soraPrompt,     // 多镜头文本
       referenceImageUrl: URL,           // 融合参考图
       config: {
           aspect_ratio: "16:9",         // 或 "9:16"
```

```
            duration: "10",               // 10/15/25
            hd: false                     // 或 true
        }
    })
    → 拿到 taskId: "video-abc123"

3. 轮询等待
   每 10 秒 GET 一次状态:
   → status: "processing", progress: 30 → 继续
   → status: "processing", progress: 60 → 继续
   → status: "completed", videoUrl: "https://cdn.xxx.com/video.mp4" → 完成!

   最多等 40 分钟（240次 × 10秒）

4. 拿到结果
   taskGroup.videoUrl = "https://cdn.xxx.com/video.mp4"
   taskGroup.generationStatus = "completed"
```

3 个任务组串行执行，每个可能需要 2~5 分钟。

⑦-E 创建子节点

不调用 AI。

为每个完成的视频创建 `SORA_VIDEO_CHILD` 子节点：

代码块

```
子节点1: { taskNumber: 1, soraPrompt: "...", videoUrl: "https://..." }
子节点2: { taskNumber: 2, soraPrompt: "...", videoUrl: "https://..." }
子节点3: { taskNumber: 3, soraPrompt: "...", videoUrl: "https://..." }
```

同时保存视频到本地文件系统和 IndexedDB。

数据字段全程追踪

以"分镜1: 男主推门走入房间"为例，追踪关键字段在每一步的状态：

步骤	存在什么结构里	visualDescription	splitImage	备注
④ 文本AI生成	`storyboardShots[0]`	"男主推门走入房间，环顾四周"	无	源数据
⑤ 传给图片AI			无	AI 看到的版本

	拼入 `gridPrompt`	被翻译+加景别成英文		
⑤ 产出	`storyboardGridImages[0]`	—	九宫格大图(含分镜1)	视觉化
⑥ 切割	`splitShots[0]`	"男主推门走入房间，环顾四周"（原样继承）	base64 小图	数据+图绑定
⑦ 分组	`taskGroups[0].splitShots[0]`	同上	同上	引用同一个对象
⑦ 提示词	`taskGroups[0].soraPrompt`	被 AI 重写成中文 Sora 格式	—	"全景，视平，轨道推拉，男主推门走入房间..."
⑦ 融合图	Canvas 拼图	—	小图被拼入参考图(带编号①)	角色三视图在下方
⑦ Sora API	`prompt` 参数	最终提示词文本	—	"Shot 2: 4.0s 全景，视平..."
⑦ Sora API	`images[0]` 参数	—	融合图 OSS URL	整张图(含①②+角色)

关键结论：分镜1 的描述数据从第 ④ 步生成后从未被修改，只是在不同步骤被"读取"用于不同目的。

连续性机制

同一视频内（同一任务组）

Sora 2 原生理解 `Shot 1/2/3...` 格式。它会在一个视频内部：

- 理解多个 Shot 属于同一场景
- 自动处理镜头过渡
- 保持人物、环境、光照一致

开头的 0.5 秒黑屏空镜是给 Sora 的"渐入"起点。

跨视频（不同任务组）

任务组之间是**完全独立的 API 调用**，Sora 不知道彼此的存在。靠三个"软保证"：

保证方式	具体机制	可靠程度

风格一致	所有任务组共用同一套风格描述词（从①号节点传下来）	较强
角色锚定	融合图底部带角色三视图，Sora "看到"角色长什么样	中等
叙事连贯	提示词按剧本顺序编写，每个 Shot 有完整场景描述和对白	强

具体表现：

1. **切割后的小图不直接给 Sora**。它们先被 Canvas 拼成融合图（带编号①②、标题行、角色三视图），再上传 OSS，最后作为 `images` 参数传给 Sora API。

2. **Sora 的主要驱动是文字提示词**，不是图片。参考图只是辅助——让 Sora "看到"角色外观和场景风格，但视频的实际画面可能和九宫格里的画不完全一样。

3. **视频之间没有帧级连接**。上一个视频的最后一帧不会作为下一个视频的输入。跨视频的连续性完全靠文字描述 + 参考图让 AI "猜"着保持一致，这就是为什么称之为"软保证"。

代码块

```
API 调用时的参数:

云雾 API:
{
  "prompt": "Shot 1: 0.5s\nScene: 纯黑空镜...\n\nShot 2: 4.0s\nScene: 全景，视平...",  ← 文字（主要驱动）
  "model": "sora-2-all",
  "images": ["https://oss.aliyuncs.com/sora-reference-xxx.png"],          ← 融合参考图（辅助）
  "orientation": "landscape",
  "duration": 10,
  "size": "medium"
}
```

AI 调用总次数

AI 类型	所在步骤	调用次数	用途
文本	① 剧本大纲	1	生成大纲 + 风格
文本	② 剧本分集	1	生成分集剧本
文本	③ 提取角色名	1	识别角色
文本	③ 角色档案	2	每个角色 1 次
文本	④ 分镜脚本	1	生成分镜结构化数据
文本	⑦-B Sora 提示词	3	每个任务组 1 次
文本合计		**9 次**	
图片	③ 表情图	2	每个主角 1 次
图片	③ 三视图	2	每个主角 1 次
图片	④ 分镜预览图	6	每个分镜 1 次（可选）
图片	⑤ 九宫格图	1	1 页 9 宫格
图片合计		**11 次**	
视频	⑦-D Sora 2	3	每个任务组 1 次
视频合计		**3 次**	
总计		**23 次 AI 调用**	

代码实现架构

1. ⑦-A 按时长自动分组

不调用 AI，纯逻辑计算。

数据来源：从所有上游 STORYBOARD_SPLITTER 节点的 `splitShots` 收集分镜数据。

分组算法（贪心装箱）：

代码块

```
const maxDuration = parseInt(sora2Config.duration);  // 10、15 或 25

let currentGroup = { shotIds: [], splitShots: [], totalDuration: 0, ... };

for (const shot of allSplitShots) {
    if (currentGroup.totalDuration + shot.duration > maxDuration
```

```
        && currentGroup.shotIds.length > 0) {
        // 当前桶放不下了，封存当前桶，创建新桶
        taskGroups.push(currentGroup);
        currentGroup = { 新的空桶 };
    }
    currentGroup.splitShots.push(shot);
    currentGroup.totalDuration += shot.duration;
}
// 最后一个桶也 push 进去
taskGroups.push(currentGroup);
```

SoraTaskGroup 完整数据结构：

代码块

```
interface SoraTaskGroup {
    id: string;                     // "tg-{timestamp}-{index}"
    taskNumber: number;             // 从 1 开始
    totalDuration: number;          // 组内时长总和
    shotIds: string[];
    splitShots: SplitStoryboardShot[];

    sora2Config: {
        aspect_ratio: '16:9' | '9:16';
        duration: '10' | '15' | '25';
        hd: boolean;
    };

    soraPrompt: string;             // 生成的提示词
    promptGenerated: boolean;
    promptModified?: boolean;       // 用户是否手动修改

    referenceImage?: string;        // 融合图 URL 或 Base64
    imageFused: boolean;

    provider?: string;              // 'yunwu' | 'sutu' | 'yijiapi' | 'kie' | 'dayuapi'

    generationStatus: 'idle' | 'prompt_ready' | 'image_fused' | 'uploading'
                    | 'generating' | 'completed' | 'failed';
    soraTaskId?: string;            // API 返回的任务 ID
    progress?: number;              // 0-100
    videoUrl?: string;              // 视频下载 URL
    videoUrlWatermarked?: string;
    error?: string;
}
```

2. ⑦-B 为每个任务组构建提示词

构建器工厂：根据节点类型自动选择构建器：

- `SORA_VIDEO_GENERATOR` → `Sora2PromptBuilder`（包含黑色空镜）
- `STORYBOARD_VIDEO_GENERATOR` → `GenericPromptBuilder`（不含黑色空镜，调用 LLM 增强）

Sora2PromptBuilder（主构建器）

不调用 AI，直接拼接文本：

代码块

```
function buildBasicPrompt(shots, includeBlackScreen, blackScreenDuration) {
    let result = '';

    // 黑色空镜
    if (includeBlackScreen) {
        result += `Shot 1:
        duration: ${blackScreenDuration.toFixed(1)}s
        Scene: 纯黑空镜，无任何视觉内容

`;
    }

    // 实际分镜（从 Shot 2 开始）
    const startIndex = includeBlackScreen ? 2 : 1;
    const actualShots = shots.map((shot, index) => {
        const shotNumber = startIndex + index;
        const sceneParts: string[] = [];
        if (shot.shotSize) sceneParts.push(shot.shotSize);
        if (shot.cameraAngle) sceneParts.push(shot.cameraAngle);
        if (shot.cameraMovement) sceneParts.push(shot.cameraMovement);
        if (shot.visualDescription) sceneParts.push(shot.visualDescription);
        if (shot.visualEffects && shot.visualEffects !== '无') sceneParts.push(`[${shot.visualEffects}]`);
        if (shot.dialogue && shot.dialogue !== '无') sceneParts.push(`"${shot.dialogue}"`);
        if (shot.audioEffects && shot.audioEffects !== '无') sceneParts.push(`[${shot.audioEffects}]`);

        const scene = sceneParts.join(', ') || shot.visualDescription || '';
        return `Shot ${shotNumber}:
        duration: ${(shot.duration || 5).toFixed(1)}s
        Scene: ${scene}`;
    }).join('\n\n');
```

```

    return result + actualShots;
}
```

输出示例：

代码块

```
Shot 1:
        duration: 0.5s
        Scene: 纯黑空镜，无任何视觉内容

Shot 2:
        duration: 4.0s
        Scene: 全景，视平，轨道推拉，男主推门走入房间，环顾四周，[镜头从门口缓缓推进]，[门轴吱呀声]

Shot 3:
        duration: 3.0s
        Scene: 中景，视平，固定，女主惊讶地回头，手中针线掉落
```

降级函数 `buildSoraStoryPrompt`

当 Sora2PromptBuilder 抛出异常时使用：

代码块

```
function buildSoraStoryPrompt(shots) {
    return shots.map((shot, index) => {
        const duration = shot.duration || 5;
        const scene = shot.visualDescription || '';
        return `Shot ${index + 1}:
        duration: ${duration.toFixed(1)}sec
        Scene: ${scene}`;
    }).join('\n\n');
}
```

差异：不含黑色空镜、Scene 仅用 visualDescription、时长后缀用 `sec` 而非 `s`。

GenericPromptBuilder（非 Sora 模型）

调用 LLM 增强提示词：

- 将分镜的景别/角度/运镜/描述/对白/音效拼成结构化文本

- 发给 LLM，要求输出 `Style: xxx\n\nShot 1:\n duration: Xs\n Scene: xxx\n Dialogue: "xxx"\n SFX: xxx` 格式
- LLM 失败时回退到直接拼接格式
- 包含 `Style:` 声明行，不含黑色空镜

3. ⑦-C 图片融合

不调用 AI，Canvas 拼图。

`fuseStoryboardImages` — **分镜图网格拼接**

代码块

```
async function fuseStoryboardImages(
    splitShots: SplitStoryboardShot[],
    options: {
        layout?: 'grid' | 'horizontal' | 'vertical';
        columns?: number;       // 默认 3
        padding?: number;       // 默认 8px
        bgColor?: string;       // 默认 '#1a1a1c'
        showNumbers?: boolean;  // 默认 true
        numberSize?: number;    // 默认 16px
    }
): Promise<string>  // 返回 Base64 PNG DataURL
```

Canvas 绘制逻辑：

1. 加载所有分镜图的 `HTMLImageElement`
2. 计算单元格尺寸（取所有图片最大宽高）
3. 填充背景色 `#1a1a1c`
4. 遍历每张图片，contain 模式居中绘制
5. 在每张图左上角绘制编号标签（白字黑底矩形，序号 ①②③...）
6. 添加顶部标题行：青色（`#22d3ee`）文字 `分镜融合图 - 共 N 个镜头`

`fuseStoryboardWithCharacterViews` — **融合分镜图 + 角色三视图**

代码块

```
async function fuseStoryboardWithCharacterViews(
    splitShots: SplitStoryboardShot[],
    characterViews: { frontView?: string; sideView?: string; backView?: string; },
    options: {
        characterPosition?: 'bottom' | 'right' | 'left';  // 默认 bottom
```

```
        characterSize?: 'small' | 'medium' | 'large';     // 对应 150/200/250px
    }
): Promise<string>
```

- 先绘制分镜网格
- 在底部（或右侧/左侧）追加角色三视图区域
- 青色标题 "角色三视图"
- 如果没有角色图，直接返回分镜融合图

`fuseMultipleTaskGroups` — 批量处理

串行遍历每个任务组，调用 `fuseStoryboardImages` 融合。单组失败不中断整体。

Base64 → OSS 上传

代码块

```
// 1. 获取 OSS 配置
const ossConfig = getOSSConfig();

// 2. 上传融合图
if (fusionResult && ossConfig) {
    const fileName = `sora-reference-${tg.id}-${Date.now()}.png`;
    const imageUrl = await uploadFileToOSS(fusionResult.fusedImage, fileName, ossConfig);
    tg.referenceImage = imageUrl;
}
// 未配置 OSS 则保留 Base64
```

支持的 OSS 提供商：ImgBB（免费直连）、腾讯云 COS（后端代理）、阿里云 OSS（前端直连 PUT + HMAC-SHA1 签名）。

上传前：如果是 Base64 DataURL，先 fetch 转 Blob。非 ImgBB 提供商会通过 Canvas 重绘为 PNG（解决 WebP 等格式不被 Sora API 接受的问题）。

4. ⑦-D 提交 Sora 2 视频生成

SoraProvider 统一接口

代码块

```
interface SoraProvider {
    readonly name: string;
    readonly displayName: string;
    transformConfig(userConfig: Sora2UserConfig): ProviderSpecificConfig;
```

```
    submitTask(params, apiKey, context?): Promise<SoraSubmitResult>;
    checkStatus(taskId, apiKey, onProgress?, context?): Promise<SoraVideoResult>;
}
```

5 个已注册提供商

提供商	name	标清模型	高清模型	提交方式
速推	`sutu`	`sora2-new`	`sora2-pro`	form-urlencoded → 后端代理
云雾	`yunwu`	`sora-2-all`	`sora-2-pro-all`	JSON → 后端代理
大洋芋	`dayuapi`	—	—	JSON → 后端代理
KIE AI	`kie`	`sora-2-image-to-video`	`sora-2-pro-image-to-video`	JSON → 后端代理
一加	`yijiapi`	`sora-2-yijia`	`sora-2-pro-25s-yijia`	JSON → 直连

API Key 和提供商选择从 localStorage 读取，默认 `sutu`。

`generateSoraVideo` — 单个任务组完整流程

代码块

```
async function generateSoraVideo(taskGroup, onProgress?, context?): Promise<SoraVideoResult>
```

执行步骤：

1. **检查是否已提交**：有 `soraTaskId` 且 `generating` 状态则跳到轮询
2. **准备参考图**：Base64 → 上传 OSS → 得到 URL；未配置 OSS 则报错
3. **提交任务**：
   - 获取提供商 → 构建 `{ prompt, referenceImageUrl, config }` 参数
   - 调用 `provider.submitTask(params, apiKey)`
   - 获取 `soraTaskId`
4. **轮询等待**：`pollSoraTaskUntilComplete(soraTaskId, onProgress, 5000)`

`pollSoraTaskUntilComplete` — 轮询逻辑

- 最大尝试次数：240 次
- 实际间隔：5000ms（5 秒），超时 = 240 × 5 秒 = 20 分钟
- 完成状态判断：`completed` / `succeeded` / `success` / `done` / `finished`（任一匹配）
- 错误状态判断：`error` / `failed` / `failure`（任一匹配）
- **只有状态完成且有 videoUrl 才返回成功**
- 超时后最后查一次，仍无结果则报错

`generateMultipleSoraVideos` — 批量串行执行

代码块

```
for (const tg of taskGroups) {
    try {
        const result = await generateSoraVideo(tg, onProgress, context);
        results.set(tg.id, result);
    } catch (e) {
        results.set(tg.id, { status: 'error', progress: 0 });
    }
}
```

串行执行，单个失败不中断。返回 `Map<taskGroupId, SoraVideoResult>`。

5. ⑦-E 创建子节点

为每个完成的任务组创建 `SORA_VIDEO_CHILD` 子节点：

代码块

```
const childNode = {
    id: `n-sora-child-${Date.now()}-${childIndex}`,
    type: 'SORA_VIDEO_CHILD',
    title: `任务组 ${taskGroup.taskNumber}`,
    status: result.status === 'completed' ? 'SUCCESS' : 'ERROR',
    data: {
        taskGroupId: taskGroup.id,
        taskNumber: taskGroup.taskNumber,
        soraPrompt: taskGroup.soraPrompt,
        videoUrl: result.videoUrl,
        duration: result.duration,
        quality: result.quality,
        isCompliant: result.isCompliant,
        soraTaskId: result.taskId
    },
```

```
    inputs: [node.id]    // 连接到父 SORA_VIDEO_GENERATOR 节点
};
```

子节点纵向排列在父节点右侧，每个偏移 150px。

6. 存储字段映射

SORA_VIDEO_GENERATOR 节点：

字段	内容	下游使用
`taskGroups`	SoraTaskGroup 数组	节点 UI 和子操作
`sora2Config`	Sora2UserConfig	分组桶容量

SoraTaskGroup 关键字段：

字段	说明
`soraPrompt`	生成的多镜头提示词
`referenceImage`	融合参考图 URL（OSS）或 Base64
`videoUrl`	生成的视频 URL
`generationStatus`	状态流转：idle → prompt_ready → image_fused → uploading → generating → completed/failed

SORA_VIDEO_CHILD 子节点：

字段	说明
`taskGroupId`	关联的任务组
`videoUrl`	视频播放 URL
`soraPrompt`	使用的提示词
`videoFilePath`	本地文件路径（用户手动保存后）

Sora 2 API 提供商

项目通过 5 家国内第三方中转服务调用 Sora 2（Sora 2 官方 API 不直接对个人开发者开放）：

提供商	标清模型	高清模型	提交方式
云雾 API	sora-2-all	sora-2-pro-all	JSON → 后端代理
速创 API	sora2-new	sora2-pro	form-urlencoded → 后端代理
一加 API	sora-2-yijia	sora-2-pro-25s-yijia	JSON → 直连
KIE	sora-2-image-to-video	sora-2-pro-image-to-video	JSON → 后端代理
大鱼 API	(类似)	(类似)	JSON → 后端代理

统一通过 `SoraProvider` 接口抽象，每家只需实现 `submitTask()` 和 `checkStatus()` 两个方法。多家中转的原因是这些服务经常不稳定/限速/涨价，多备几家可以互相降级。